

ETHENA · SECURITY REVIEW

# Ethena

# Pay Updates.

A security review of Pay Updates.

| CLIENT | PROJECT     | AUDITORS | AUDIT WINDOW            | FINAL REPORT |
|--------|-------------|----------|-------------------------|--------------|
| Ethena | Pay Updates | Guardian | June 10 – June 29, 2026 | July 6, 2026 |

§ 00

# Contents.

128 pages — overview, confidence ranking, scope, 81 findings, disclaimer, and about Guardian.

|      |                                  |        |
|------|----------------------------------|--------|
| § 01 | Overview                         | P. 03  |
| § 02 | Confidence Ranking               | P. 04  |
| § 03 | Scope & Classifications          | P. 05  |
|      | Files in Scope                   | P. 06  |
| § 04 | Findings                         | P. 07  |
|      | Main Review · 53 findings        | P. 13  |
|      | Remediation Review · 28 findings | P. 87  |
| § 05 | Disclaimer                       | P. 127 |
| § 06 | About Guardian                   | P. 128 |

§ 01

# Overview.

Guardian reviewed Pay Updates across 2 review rounds. The final report includes 3 High, 14 Medium, 39 Low, 25 Informational findings.

|                         |                                                            |
|-------------------------|------------------------------------------------------------|
| AUDIT FIRM              | METHODOLOGY                                                |
| Guardian                | Smart contract · Manual · Main Review · Remediation Review |
| CLIENT                  |                                                            |
| Ethena                  |                                                            |
| PROJECT                 |                                                            |
| Pay Updates             |                                                            |
| PREPARED BY             |                                                            |
| Guardian                |                                                            |
| AUDIT WINDOW            |                                                            |
| June 10 – June 29, 2026 |                                                            |
| FINAL REPORT            |                                                            |
| July 6, 2026            |                                                            |

## 1.1 Confidence Ranking

3 / 5

Moderate Confidence

Guardian assigns a Confidence Ranking of 3 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

|                     |                |                     |                 |                      |
|---------------------|----------------|---------------------|-----------------|----------------------|
| 1                   | 2              | 3                   | 4               | 5                    |
| VERY LOW CONFIDENCE | LOW CONFIDENCE | MODERATE CONFIDENCE | HIGH CONFIDENCE | VERY HIGH CONFIDENCE |

## 1.2 Findings at a Glance

| SEVERITY       | TOTAL | PENDING | ACK | PARTIAL | RESOLVED |
|----------------|-------|---------|-----|---------|----------|
| Critical       | 0     | 0       | 0   | 0       | 0        |
| High           | 3     | 0       | 0   | 1       | 2        |
| Medium         | 14    | 0       | 5   | 2       | 7        |
| Low            | 39    | 0       | 22  | 5       | 12       |
| Informational  | 25    | 0       | 18  | 3       | 4        |
| All severities | 81    | 0       | 45  | 11      | 25       |

## § 02

# Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

| CONFIDENCE RANKING                              | DEFINITION AND RECOMMENDATION                                                                                                                                                                                               | RISK PROFILE                                                                              |
|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| <b>5 Very High Confidence</b>                   | Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit. | 0 High/Critical findings and few Low/Medium severity findings.                            |
| <b>4 High Confidence</b>                        | Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.    | 0-1 High/Critical findings per engagement week and little to no Medium severity findings. |
| <b>3 Moderate Confidence</b><br><b>ASSIGNED</b> | Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.                                                            | Some High/Critical findings, with various Medium severity findings.                       |
| <b>2 Low Confidence</b>                         | Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.                                                                     | Over 6 High and Critical findings across the engagement.                                  |
| <b>1 Very Low Confidence</b>                    | The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.                                                                           | Over 12 High and Critical findings across the engagement.                                 |

§ 03

# Scope & Classifications.

Guardian reviewed Pay Updates.

## 3.1 Severity Matrix

|            | IMP · HIGH | IMP · MED | IMP · LOW |
|------------|------------|-----------|-----------|
| LIK · HIGH | Critical   | High      | Medium    |
| LIK · MED  | High       | Medium    | Low       |
| LIK · LOW  | Medium     | Low       | Low       |

IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW

Unlikely to ever occur in production.

# Files in Scope.

| FILE                             | NSLOC | TOTAL |
|----------------------------------|-------|-------|
| No files were recorded in scope. |       |       |

## § 04

# Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

| ID          | TITLE                                            | CATEGORY            | SEVERITY | STATUS             | PAGE  |
|-------------|--------------------------------------------------|---------------------|----------|--------------------|-------|
| MAIN REVIEW |                                                  |                     |          |                    |       |
| H-01        | Target signer can cancel its own removal         | Validation          | HIGH     | RESOLVED           | p. 13 |
| H-02        | Pause blocks revoke, not ERC-1271 fills          | Unexpected Behavior | HIGH     | PARTIALLY RESOLVED | p. 15 |
| H-03        | Pending-removal signer can install a replacement | Unexpected Behavior | HIGH     | RESOLVED           | p. 16 |
| M-01        | Preauthorized ERC1271 hashes ignore deadlines    | Validation          | MEDIUM   | RESOLVED           | p. 18 |
| M-02        | Direct spends bypass recipient policy            | Unexpected Behavior | MEDIUM   | PARTIALLY RESOLVED | p. 19 |
| M-03        | ProxyAdmin upgrade can bypass beacon timelock    | Unexpected Behavior | MEDIUM   | RESOLVED           | p. 21 |
| M-04        | Pending recovery stuck by executor rotation      | Unexpected Behavior | MEDIUM   | RESOLVED           | p. 22 |
| M-05        | Stale recoveries remain executable               | Unexpected Behavior | MEDIUM   | RESOLVED           | p. 23 |
| M-06        | Re-added signers can reuse old unused signatures | Unexpected Behavior | MEDIUM   | RESOLVED           | p. 24 |
| M-07        | Pre-authorized hashes survive signer removal     | Validation          | MEDIUM   | PARTIALLY RESOLVED | p. 25 |
| M-08        | Live ERC-1271 bypasses executor policy           | Unexpected Behavior | MEDIUM   | RESOLVED           | p. 26 |
| M-09        | Emergency blocks Permit2 cleanup and CoW cancel  | Unexpected Behavior | MEDIUM   | RESOLVED           | p. 29 |

Findings continued.

| ID   | TITLE                                            | CATEGORY            | SEVERITY | STATUS             | PAGE  |
|------|--------------------------------------------------|---------------------|----------|--------------------|-------|
| M-10 | Recovery-admin backdoor survives signer removal  | Access Control      | MEDIUM   | ACKNOWLEDGED       | p. 31 |
| L-01 | BatchExecutor cannot be factory executor         | Unexpected Behavior | LOW      | ACKNOWLEDGED       | p. 33 |
| L-02 | Permissionless nonce invalidation censors spends | DoS                 | LOW      | RESOLVED           | p. 34 |
| L-03 | Reused orderHash can strand old allowances       | Unexpected Behavior | LOW      | ACKNOWLEDGED       | p. 35 |
| L-04 | Old orders use newer same-spender allowance      | Validation          | LOW      | RESOLVED           | p. 36 |
| L-05 | Pending signer ready views ignore expiry         | Validation          | LOW      | PARTIALLY RESOLVED | p. 38 |
| L-06 | Direct mode cannot recover received NFTs         | Unexpected Behavior | LOW      | RESOLVED           | p. 39 |
| L-07 | Executor calls create untracked approvals        | Trust Assumptions   | LOW      | PARTIALLY RESOLVED | p. 40 |
| L-08 | Empty recovery reset keeps opt-out active        | Unexpected Behavior | LOW      | RESOLVED           | p. 42 |
| L-09 | BatchExecutor owner can freeze config            | Unexpected Behavior | LOW      | RESOLVED           | p. 43 |
| L-10 | Upgrade script assumes EOA ProxyAdmin owner      | Configuration       | LOW      | RESOLVED           | p. 44 |
| L-11 | Batch policy misses executor withdrawals         | Validation          | LOW      | PARTIALLY RESOLVED | p. 45 |
| L-12 | Dual-sign ops are not bound to executor          | Validation          | LOW      | PARTIALLY RESOLVED | p. 46 |
| L-13 | Scheduled governance actions never expire        | Unexpected Behavior | LOW      | RESOLVED           | p. 48 |
| L-14 | Executor swap can brick a pre-auth order         | Unexpected Behavior | LOW      | RESOLVED           | p. 50 |
| L-15 | Gas guard can still revert batches               | Unexpected Behavior | LOW      | PARTIALLY RESOLVED | p. 52 |

## Findings continued.

| ID   | TITLE                                            | CATEGORY       | SEVERITY      | STATUS             | PAGE  |
|------|--------------------------------------------------|----------------|---------------|--------------------|-------|
| L-16 | Executor rotation can overlap wallet roles       | Validation     | LOW           | RESOLVED           | p. 54 |
| L-17 | externalApprove silently bricks live orders      | Logical Error  | LOW           | RESOLVED           | p. 56 |
| L-18 | Silent batch truncation is under-observable      | Events         | LOW           | RESOLVED           | p. 58 |
| L-19 | Cancel signature not bound to its recovery       | Signatures     | LOW           | RESOLVED           | p. 59 |
| L-20 | Pending passkey removal view returns stale index | Validation     | LOW           | ACKNOWLEDGED       | p. 61 |
| I-01 | Default daily limits allow duplicate arrays      | Validation     | INFORMATIONAL | PARTIALLY RESOLVED | p. 63 |
| I-02 | WebAuthn RP ID hash is not enforced on-chain     | Warning        | INFORMATIONAL | PARTIALLY RESOLVED | p. 64 |
| I-03 | Batch accepts non-wallet call targets            | Warning        | INFORMATIONAL | ACKNOWLEDGED       | p. 65 |
| I-04 | Policy reverts abort entire batch                | Warning        | INFORMATIONAL | ACKNOWLEDGED       | p. 66 |
| I-05 | Recovery admins cannot replace pending recovery  | Warning        | INFORMATIONAL | ACKNOWLEDGED       | p. 67 |
| I-06 | External approval docs omit approvalDeadline     | Documentation  | INFORMATIONAL | ACKNOWLEDGED       | p. 69 |
| I-07 | Query guides use nonexistent view calls          | Documentation  | INFORMATIONAL | ACKNOWLEDGED       | p. 70 |
| I-08 | Runbook uses bytes16 wallet UUID selectors       | Documentation  | INFORMATIONAL | ACKNOWLEDGED       | p. 71 |
| I-09 | Batch docs use removed event name                | Documentation  | INFORMATIONAL | ACKNOWLEDGED       | p. 72 |
| I-10 | Threat model overstates direct-mode caller risk  | Documentation  | INFORMATIONAL | ACKNOWLEDGED       | p. 73 |
| I-11 | Overloaded DailyLimitExceeded revert error       | Best Practices | INFORMATIONAL | ACKNOWLEDGED       | p. 75 |

## Findings continued.

| ID   | TITLE                                          | CATEGORY            | SEVERITY      | STATUS             | PAGE  |
|------|------------------------------------------------|---------------------|---------------|--------------------|-------|
| I-12 | Passkey signer docs use qx instead of key hash | Documentation       | INFORMATIONAL | ACKNOWLEDGED       | p. 76 |
| I-13 | Docs claim delay is signed in config           | Documentation       | INFORMATIONAL | ACKNOWLEDGED       | p. 77 |
| I-14 | Parent preauth can authorize child ops         | Unexpected Behavior | INFORMATIONAL | ACKNOWLEDGED       | p. 78 |
| I-15 | Recovery bypasses signer count caps            | DoS                 | INFORMATIONAL | ACKNOWLEDGED       | p. 80 |
| I-16 | Upgrade omits approval view selectors          | Configuration       | INFORMATIONAL | RESOLVED           | p. 81 |
| I-17 | executeRecovery breaks reentrancy invariant    | Informational       | INFORMATIONAL | RESOLVED           | p. 82 |
| I-18 | Executor wallet preauth satisfies executor sig | Warning             | INFORMATIONAL | RESOLVED           | p. 83 |
| I-19 | BatchExecutor deploy ignores initial owner     | Documentation       | INFORMATIONAL | PARTIALLY RESOLVED | p. 85 |
| I-20 | Recovery reset event misreports admin set      | Best Practices      | INFORMATIONAL | RESOLVED           | p. 86 |

## REMEDIATION REVIEW

|      |                                                 |                     |        |              |       |
|------|-------------------------------------------------|---------------------|--------|--------------|-------|
| M-01 | Authorized deployer seizes funded UUID wallets  | Unexpected Behavior | MEDIUM | ACKNOWLEDGED | p. 87 |
| M-02 | Prefunded wallets inherit mutable defaults      | Unexpected Behavior | MEDIUM | ACKNOWLEDGED | p. 90 |
| M-03 | Preauthorized hashes ignore pending removal     | Validation          | MEDIUM | ACKNOWLEDGED | p. 92 |
| M-04 | Approve and call can leave approvals            | Trust Assumptions   | MEDIUM | ACKNOWLEDGED | p. 94 |
| L-01 | Immediate RP ID hash update can brick passkeys  | Compatibility       | LOW    | ACKNOWLEDGED | p. 95 |
| L-02 | Old executor keeps deployer role after rotation | Trust Assumptions   | LOW    | ACKNOWLEDGED | p. 96 |

## Findings continued.

| ID   | TITLE                                            | CATEGORY            | SEVERITY | STATUS       | PAGE   |
|------|--------------------------------------------------|---------------------|----------|--------------|--------|
| L-03 | Emergency revokes require executor epoch         | Unexpected Behavior | LOW      | ACKNOWLEDGED | p. 98  |
| L-04 | Batch executor can disable blocklist             | Unexpected Behavior | LOW      | ACKNOWLEDGED | p. 100 |
| L-05 | Pending recovery initiator can become executor   | Validation          | LOW      | ACKNOWLEDGED | p. 102 |
| L-06 | Default recovery admins can become unusable      | Validation          | LOW      | ACKNOWLEDGED | p. 104 |
| L-07 | Allowance spends check relayer for removal       | Unexpected Behavior | LOW      | ACKNOWLEDGED | p. 105 |
| L-08 | Recipient blocklist is not wallet-wide           | Access Control      | LOW      | ACKNOWLEDGED | p. 106 |
| L-09 | ProxyAdmin delay check can brick the kill switch | DoS                 | LOW      | ACKNOWLEDGED | p. 108 |
| L-10 | Batch subcall gas can starve later items         | Unexpected Behavior | LOW      | ACKNOWLEDGED | p. 110 |
| L-11 | Stale exits end later direct-mode sessions       | Unexpected Behavior | LOW      | ACKNOWLEDGED | p. 111 |
| L-12 | No-op recovery config cancels pending recovery   | Unexpected Behavior | LOW      | ACKNOWLEDGED | p. 112 |
| L-13 | Beacon wallets reject 2300-gas native transfers  | Unexpected Behavior | LOW      | ACKNOWLEDGED | p. 113 |
| L-14 | ECDSA bypasses code-owner ERC-1271 policy        | Validation          | LOW      | ACKNOWLEDGED | p. 114 |
| L-15 | Pause probe blocks direct-mode signer changes    | Validation          | LOW      | ACKNOWLEDGED | p. 115 |
| L-16 | Stale pending owner additions outlive removal    | Unexpected Behavior | LOW      | ACKNOWLEDGED | p. 117 |
| L-17 | Same-block direct-mode cancels collide           | Unexpected Behavior | LOW      | ACKNOWLEDGED | p. 118 |
| L-18 | Zero-coordinate P-256 passkeys are rejected      | Validation          | LOW      | ACKNOWLEDGED | p. 120 |

## Findings continued.

| ID   | TITLE                                            | CATEGORY            | SEVERITY      | STATUS       | PAGE   |
|------|--------------------------------------------------|---------------------|---------------|--------------|--------|
| L-19 | Zero externalApprove records ghost owner         | Unexpected Behavior | LOW           | ACKNOWLEDGED | p. 121 |
| I-01 | Emergency revocations emit no events             | Events              | INFORMATIONAL | ACKNOWLEDGED | p. 122 |
| I-02 | Direct NFT transfers lack target checks          | Warning             | INFORMATIONAL | ACKNOWLEDGED | p. 123 |
| I-03 | Base64URL challenge encoding overflows output    | Unexpected Behavior | INFORMATIONAL | ACKNOWLEDGED | p. 124 |
| I-04 | Factory init optimizes out EIP-1153 probe        | Unexpected Behavior | INFORMATIONAL | ACKNOWLEDGED | p. 125 |
| I-05 | Batch partial failures lack item log attribution | Events              | INFORMATIONAL | ACKNOWLEDGED | p. 126 |

HIGH **H-01**

RESOLVED

# Target signer can cancel its own removal

CATEGORY Validation  
LOCATION src/facets/AuthFacet.sol  
REVIEW Main Review

## DESCRIPTION

`cancelRemoveOwner` accepts a cancellation signature from any registered signer. It then calls `LibAuth.cancelRemoveOwner(actionId)` without checking whether the verified `signerKey` is the owner pending removal.

This means the owner being removed can sign `CancelRemoveOwner` while it is still registered. If that signer is compromised, the attacker can cancel its own pending removal during the 24 hour cooldown. The honest signer can start removal again, but the attacker can keep cancelling each pending action as long as the compromised signer remains registered.

`cancelRemovePasskey` has the same missing exclusion before it calls `LibAuth.cancelRemovePasskey(actionId)`. Therefore a compromised passkey can also cancel a pending removal for its own coordinates.

This does not give an outsider access to a wallet. The attacker must already control a registered signer. However, the protocol relies on signer removal to contain a stolen or phished signer key. Because the target signer can cancel each pending removal, the compromised signer can remain registered and keep authorizing wallet actions, including direct mode operations and direct withdrawals once direct mode is active.

## RECOMMENDATION

Do not fix this with a blanket rule that the removal target can never cancel. That rule protects against a compromised signer blocking its own removal, but it creates the opposite failure for two-signer wallets: a compromised signer could initiate removal of the honest signer and the honest signer would be unable to cancel the pending removal.

Instead, treat signer removal as a contested action. Store the signer that initiated the removal and add explicit conflict handling before `LibAuth.cancelRemoveOwner(actionId)` or `LibAuth.cancelRemovePasskey(actionId)` deletes the pending action.

For wallets with three or more signers, require cancellation or final execution to be authorized by a signer that is neither the removal target nor the initiator, or require a small quorum. For two-signer wallets, do not allow one signer to unilaterally remove the other

**H-01** Target signer can cancel its own removal

CONTINUED

without recovery-admin, guardian, or stronger multi-factor involvement. If the removal target objects, move the action into a contested state instead of simply deleting it or letting either party unilaterally win.

Also quarantine pending-removal signers from security-sensitive signer-management actions, such as approving new signers or cancelling direct mode, unless the action is part of the contested-removal resolution. This keeps the compromised signer from preserving control through a replacement key while still avoiding a design where the honest target cannot defend against malicious removal.

**— RESOLUTION**

Status: Resolved.

HIGH **H-02**

PARTIALLY RESOLVED

# Pause blocks revoke, not ERC-1271 fills

|          |                             |
|----------|-----------------------------|
| CATEGORY | Unexpected Behavior         |
| LOCATION | src/facets/ERC1271Facet.sol |
| REVIEW   | Main Review                 |

## DESCRIPTION

When the factory's global pause is active, `LibDirectMode.enforceUserNotBlocked()` reverts if the wallet is not already in direct mode. `ExternalApproveAndPreAuthorizeHashFacet.revokePreAuthorizedHash` calls `enforceUserNotBlocked`, so a user who needs to cancel a Fusion+ pre-authorization during a global emergency cannot do so unless they had already entered direct mode before the pause. The alternative cleanup path, `clearStaleApproval`, has no pause check but requires `block.timestamp >= entry.expiresAt`, meaning it is only callable after the `approvalDeadline` has already passed. Meanwhile, `ERC1271Facet.isValidSignature` is a view function with no pause guard and returns `MAGIC_VALUE` for any pre-authorized hash regardless of pause state. The ERC-20 approval granted by `externalApproveAndPreAuthorizeHash` lives on the token contract, entirely outside the wallet's pause domain. A Fusion+ resolver can therefore successfully execute a fill (token `transferFrom` plus `isValidSignature` magic value) during a global pause at prices the user can no longer cancel. The user's 1-hour direct-mode cooldown exists but provides no protection within that window.

During a global pause, users cannot revoke pre-authorized Fusion+ orders (`revokePreAuthorizedHash` is paused-blocked) while resolvers can still fill them (`isValidSignature` and ERC-20 allowances are not paused). Users may lose funds at stale prices during an emergency pause window.

## RECOMMENDATION

Add a pause check inside `ERC1271Facet.isValidSignature` for pre-authorized hashes: if the wallet's factory is globally paused, return `ERC1271_INVALID` (or revert) for pre-authorized hash lookups, preventing fills during emergencies. Additionally, consider making `revokePreAuthorizedHash` available without the `enforceUserNotBlocked` gate (treat it like `cancelRecovery`, always accessible, since it is purely protective), or at minimum bypass the pause gate for single-sig revocations the way direct-mode operations bypass it.

## RESOLUTION

Status: Partially resolved.

HIGH **H-03**

RESOLVED

# Pending-removal signer can install a replacement

|          |                          |
|----------|--------------------------|
| CATEGORY | Unexpected Behavior      |
| LOCATION | src/facets/AuthFacet.sol |
| REVIEW   | Main Review              |

## DESCRIPTION

`approveAddOwner` accepts a signature from any signer that is still registered. It does not check whether that signer is already pending removal. Therefore, a compromised owner or passkey can keep approving signer-management actions during the cooldown that is supposed to contain it.

For example, assume Alice has a wallet with an honest passkey and an address owner whose private key has been phished. Alice uses the passkey to initiate removal of the compromised owner. During the 24 hour removal cooldown, the attacker uses the compromised owner to initiate adding a new attacker-controlled owner. Once the addition cooldown has elapsed, the attacker approves that pending addition with the compromised owner. Alice can still execute the original removal, but only the old compromised owner is removed. The replacement owner remains registered and can continue authorizing wallet operations.

This is different from allowing a signer to cancel its own removal. Even if signer-removal cancellation is redesigned as a contested action, the attacker can preserve access through a replacement signer unless pending-removal signers are restricted from approving new signer-management actions. The removal succeeds, but it no longer removes the attacker's control of the wallet.

## RECOMMENDATION

Treat pending removal as an authorization quarantine for actions that can preserve or expand control. Once an owner or passkey is queued for removal, that signer should not be able to initiate or approve new signers, approve replacement keys, change recovery configuration, or perform other security-sensitive signer-management actions.

## H-03 Pending-removal signer can install a replacement

CONTINUED

At minimum, make `initiateAddOwner`, `approveAddOwner`, `initiateAddPasskey` and `approveAddPasskey` reject signatures from signer keys that are currently pending removal. A stronger fix is to centralize this check immediately after `LibAuth.verifyOwnerSignature` returns the signer key, then apply it to every signer-management function where a pending-removal signer should no longer be trusted.

Do not use this quarantine to prevent the removal target from objecting to its own removal in every case. That creates the opposite two-signer failure where a compromised signer can initiate removal of the honest signer and the honest signer cannot stop it. Cancellation and objection should follow the contested-removal design: with three or more signers, require a third signer or quorum; with two signers, require recovery-admin, guardian, or another stronger recovery mechanism to resolve the dispute.

### — RESOLUTION

Status: Resolved.

MEDIUM **M-01**

RESOLVED

# Preauthorized ERC1271 hashes ignore deadlines

CATEGORY Validation  
LOCATION src/facets/ERC1271Facet.sol  
REVIEW Main Review

## DESCRIPTION

`ERC1271Facet.isValidSignature` returns the ERC-1271 magic value whenever `LibPreAuthorizedHash.isAuthorized(hash)` is true. It does not check an expiry timestamp before accepting the hash.

`externalApproveAndPreAuthorizeHash` accepts an `approvalDeadline`, but that deadline is only stored for later stale-approval cleanup. The preauthorized hash itself is stored as a boolean in `LibPreAuthorizedHash`, so it remains valid until a separate cleanup or revocation transaction clears it.

Consequently, an external protocol that relies on the wallet's ERC-1271 response can continue treating an expired order or intent hash as valid. If the related token approval has not been cleared yet, stale settlement can remain possible after the wallet-level approval deadline.

## RECOMMENDATION

Store an expiry timestamp with each preauthorized hash and make `isValidSignature` return invalid after that timestamp. The expiry should be written when `externalApproveAndPreAuthorizeHash` authorizes the hash.

When an expired hash is detected, either reject it without changing state or expose a cleanup function that clears both the token approval metadata and the ERC-1271 authorization. Avoid keeping the ERC-1271 authorization valid after the approval deadline has passed.

## RESOLUTION

Status: Resolved.

MEDIUM **M-02**

PARTIALLY RESOLVED

# Direct spends bypass recipient policy

|          |                               |
|----------|-------------------------------|
| CATEGORY | Unexpected Behavior           |
| LOCATION | src/facets/AllowanceFacet.sol |
| REVIEW   | Main Review                   |

## DESCRIPTION

`BatchExecutor.batchExecute` checks `blocked` and `approvedRecipients` before it forwards `spendDailyAllowance` or `spendDailyAllowanceEth` calldata to a wallet. Those checks only run inside `BatchExecutor`.

The wallet spend functions are still externally callable. `spendDailyAllowance` verifies the executor signature over `(token, to, amount, spendNonce, deadline)`, consumes the spend nonce, checks the daily limit and transfers tokens to `to`. It does not require `msg.sender` to be `BatchExecutor`. It also does not check the `BatchExecutor` allowlist or blocklist before transferring funds.

For example, assume the executor signs a card spend to a merchant and that merchant is later blocklisted before settlement. If a relayer submits the spend through `BatchExecutor`, the batch contract rejects the blocked recipient. The same signed calldata can still be sent directly to the wallet proxy. The wallet accepts the executor signature and transfers funds to the blocked merchant because the recipient policy is not enforced where the transfer happens.

Consequently, the configured recipient policy is optional. It protects only spends routed through `BatchExecutor`, not all executor-signed allowance spends.

## RECOMMENDATION

Enforce recipient policy in `spendDailyAllowance` and `spendDailyAllowanceEth`, since those functions perform the actual token and ETH transfers.

If recipient policy must remain in a gateway contract, require allowance spend functions to be called by the configured gateway and bind that gateway address into the signed spend digest. Do not rely on `BatchExecutor` checks unless direct wallet calls are blocked or enforce the same policy.

M-02 Direct spends bypass recipient policy

CONTINUED

— RESOLUTION

Status: Partially resolved.

MEDIUM **M-03**

RESOLVED

# ProxyAdmin upgrade can bypass beacon timelock

CATEGORY Unexpected Behavior  
LOCATION src/UserProxyFactoryBeacon.sol  
REVIEW Main Review

## DESCRIPTION

The wallet beacon is owned by the factory proxy address. The normal factory functions enforce a 48-hour delay by requiring `scheduleUpgrade` first and `executeUpgrade` after `UPGRADE_DELAY`.

The factory proxy is itself upgradeable through `ProxyAdmin.upgradeAndCall`. A replacement factory implementation can run code in the proxy's storage context during that same call. Since the proxy address is the beacon owner, the replacement implementation can call `beacon.upgradeTo` immediately and change the wallet implementation without using the factory's 48-hour beacon-upgrade schedule.

This is not a public exploit as it requires `ProxyAdmin` authority or a governance-approved factory implementation upgrade. However, it still weakens the documented upgrade model because the docs describe the `ProxyAdmin` timelock and the factory's internal 48-hour delay as layered protection for wallet implementation changes. In practice, a factory implementation upgrade can make the wallet beacon change effective after only the `ProxyAdmin` delay.

## RECOMMENDATION

Treat factory implementation upgrades as full wallet-system upgrades. Set the `ProxyAdmin` timelock delay to at least the factory `UPGRADE_DELAY` and document that a factory upgrade can affect the wallet beacon immediately. For stronger isolation, move beacon ownership to a non-upgradeable timelock-owned controller. Another option is to require a separate on-chain check before any factory implementation can upgrade the beacon.

## RESOLUTION

Status: Resolved.

MEDIUM **M-04**

RESOLVED

# Pending recovery stuck by executor rotation

CATEGORY Unexpected Behavior  
LOCATION src/facets/RecoveryFacet.sol  
REVIEW Main Review

## DESCRIPTION

`executeRecovery` clears the pending recovery and then calls `LibAuth.addOwnerDirect` for address signers. `addOwnerDirect` rechecks the current factory executor and reverts if the pending owner is now the executor.

If the factory executor changes during the recovery delay and becomes the pending owner, `executeRecovery` reverts. The earlier `clearPendingRecovery` call is rolled back with the revert, so the invalid pending recovery remains active. Since `initiateRecovery` rejects any new recovery while `pendingActive` is true, the recovery admin cannot start a replacement recovery. Only an existing wallet owner can cancel the stale recovery.

This is a bad failure mode for recovery because the feature is specifically needed when the user may not have access to an existing signer.

## RECOMMENDATION

Handle invalid pending address recoveries without leaving the wallet stuck. If the pending owner equals the current executor at execution time, clear the pending recovery and emit an invalidation event instead of reverting. Alternatively, allow recovery admins to cancel or replace a pending recovery once it has become invalid or after the recovery delay has elapsed.

## RESOLUTION

Status: Resolved.

MEDIUM **M-05**

RESOLVED

## Stale recoveries remain executable

|          |                              |
|----------|------------------------------|
| CATEGORY | Unexpected Behavior          |
| LOCATION | src/facets/RecoveryFacet.sol |
| REVIEW   | Main Review                  |

### — DESCRIPTION

`initiateRecovery` stores only whether a recovery is pending, the new signer and the initiation timestamp. It does not store the initiating recovery admin, the recovery-admin configuration hash or an expiry time.

`executeRecovery` then checks only that a recovery is pending and that the 10-day minimum delay has elapsed.

Consequently, removing a recovery admin does not invalidate a recovery that the admin already initiated. The pending recovery also remains executable indefinitely after the delay, even though the security documentation says pending timelocked actions, including recovery, expire after 7 days.

For example, a default recovery admin is compromised and initiates recovery to add an attacker-controlled signer. Governance later removes that admin from the default recovery admin list, but the old pending recovery remains valid. If the wallet owner cannot cancel it with an existing signer, anyone can execute the stale recovery after the delay and add the attacker-controlled signer.

### — RECOMMENDATION

Store the recovery initiator, the recovery-admin configuration hash and an expiry timestamp when recovery is initiated. At execution time, require the stored initiator to still be an effective recovery admin under the same configuration. Reject pending recoveries that are past their expiry and consider cancelling affected pending recoveries when factory default recovery admins change.

### — RESOLUTION

Status: Resolved.

MEDIUM **M-06**

RESOLVED

# Re-added signers can reuse old unused signatures

CATEGORY Unexpected Behavior  
LOCATION `src/libraries/LibAuth.sol`  
REVIEW Main Review

## — DESCRIPTION

Signer nonces are keyed by owner address or passkey coordinates. Removing an owner or passkey deletes it from the active signer array, but the removal does not invalidate the signer's nonce or otherwise mark a new signer generation.

If the same signer is later re-added, its old nonce value is still present. Any old signature that was created before removal, used that still-current nonce and has not expired can become valid again after re-addition.

This can resurrect stale approvals, withdrawals or arbitrary-call authorizations across signer lifecycle changes. The signed parameters cannot be changed, but the user may have expected removal to invalidate all outstanding authority from that signer.

## — RECOMMENDATION

Invalidate signer authorization state when a signer is removed. A simple fix is to increment `signerNonces[removedSignerKey]` during owner and passkey removal. A stronger fix is to add a signer generation counter and include it in every signed struct, so removing and re-adding the same address or passkey starts a fresh authorization generation.

## — RESOLUTION

Status: Resolved.

MEDIUM **M-07**

PARTIALLY RESOLVED

# Pre-authorized hashes survive signer removal

CATEGORY Validation  
LOCATION `src/libraries/LibPreAuthorizedHash.sol`  
REVIEW Main Review

## DESCRIPTION

Pre-authorized hashes are stored as wallet-wide booleans. The storage does not record which signer authorized the hash and `ERC1271Facet.isValidSignature` returns the ERC-1271 magic value for any authorized hash without checking current signer membership.

Consequently, removing a compromised signer does not invalidate hashes that signer already pre-authorized. If the authorization also created an ERC-20 allowance, the spender can keep using the pre-authorized order and allowance after the signer has been removed from the wallet.

This is distinct from stale live signatures: live signatures from a removed signer fail because the signer is no longer registered. Pre-authorized hashes bypass live signer verification entirely.

## RECOMMENDATION

Store signer attribution and a signer epoch with each pre-authorized hash. `isValidSignature` should reject pre-authorized hashes whose authorizing signer is no longer registered or whose signer epoch changed after authorization.

Also add a wallet-wide pre-authorization epoch or a signer-scoped revocation mechanism so users can invalidate all pre-authorized hashes created by a compromised signer during signer rotation.

## RESOLUTION

Status: Partially resolved.

MEDIUM **M-08**

RESOLVED

# Live ERC-1271 bypasses executor policy

CATEGORY Unexpected Behavior  
LOCATION src/facets/ERC1271Facet.sol  
REVIEW Main Review

## DESCRIPTION

`ERC1271Facet.isValidSignature` accepts any live signature from a registered wallet signer for any hash supplied by an external protocol. It does not require executor co-signature, daily allowance checks, recipient policy, direct-mode policy or any wallet-side parsing of the external protocol payload.

The normal wallet spending model has two "locks". First, the user signer says "yes". Second, the executor says "yes". That is the intended safety model for wallet fund movement.

Permit2 changes the shape of the action. The wallet can first approve Permit2 to spend tokens. When this approval is created through the wallet's normal approval functions, it requires both the signer and the executor. After Permit2 has that token approval, however, Permit2 can move funds if it receives a valid ERC-1271 signature from the wallet.

The problem starts at `src/facets/ERC1271Facet`. `ERC1271Facet.isValidSignature` accepts any live signature from a registered wallet signer. It does not ask the executor. It does not check daily limits. It does not check recipient policy. It does not parse the Permit2 spend.

The sequence is:

1. The wallet gives Permit2 an ERC-20 allowance through an owner-and-executor approved wallet action.
2. Later, a registered signer signs a Permit2 message.
3. Permit2 asks the wallet whether the signer signature is valid.
4. The wallet answers yes through ERC-1271.
5. Permit2 spends from the wallet using the existing allowance.
6. The executor never approves that later Permit2 spend.

**M-08** Live ERC-1271 bypasses executor policy

CONTINUED

This matters for shared spenders and external settlement protocols that can consume ERC-1271 signatures after an allowance exists. Permit2 is the clearest shared-spender example. Once the wallet has granted an ERC-20 allowance to the Permit2 contract, a single registered signer can sign a Permit2

`SignatureTransfer` digest for an arbitrary Permit2 spender. That spender can then call `Permit2.permitTransferFrom` and move funds from the wallet without any executor signature on the actual spend.

The same issue exists for Permit2 `AllowanceTransfer.permit`. A registered signer can sign a Permit2 allowance permit that creates a durable `(wallet, token, spender)` sub-allowance inside Permit2. The spender can then call `Permit2.transferFrom` until the signed amount is depleted or the Permit2 expiration passes. The wallet does not register or clear that sub-allowance.

The batch variants have the same trust boundary. A single registered signer signature over `PermitBatchTransferFrom` can move multiple approved tokens through Permit2 and a single registered signer signature over `AllowanceTransfer.PermitBatch` can create multiple durable Permit2 sub-allowances. The executor still only approved the earlier wallet-to-Permit2 ERC-20 allowances, not the later Permit2 batch spend or batch sub-allowance creation.

Permit2's basic `permitTransferFrom` digest binds the token, maximum amount, spender, nonce and deadline. The final `transferDetails.to` recipient is supplied by the spender at execution time. Therefore the recipient is controlled by the signed spender, not by wallet logic. This is normal Permit2 behavior, but it is a sharp edge for this wallet because ERC-1271 live signatures are treated as signer-only authorizations.

`permitWitnessTransferFrom` verifies the witness hash, but Permit2 does not interpret the witness or compare it to `transferDetails.to`. If the product relies on a witness to bind the user-facing recipient or order details, the spender or integration contract must enforce that the witness fields match the actual transfer details. The wallet does not do that enforcement and Permit2 does not do it generically.

This does not let an outsider spend from a wallet with no valid signer signature. It also does not let the signer create the initial wallet-to-Permit2 ERC-20 approval without the executor when the normal wallet approval functions are used. The issue is the policy boundary after that approval exists: if the product expects the executor to approve all fund-moving operations, an active Permit2 allowance lets one registered signer bypass that executor review for later Permit2 transfers and Permit2 sub-allowance creation.

Cleanup is also asymmetric. Permit2 `lockdown`, `invalidateNonces` and `invalidateUnorderedNonces` use `msg.sender` as the Permit2 owner. For this smart-contract wallet, those cleanup calls must originate from the wallet, which currently means routing them through an owner-and-executor approved `executorExternalCall`. A signer-only ERC-1271 signature can

**M-08** Live ERC-1271 bypasses executor policy

CONTINUED

create a Permit2 sub-allowance, but signer-only cleanup of that sub-allowance is not available through Permit2.

CoW Protocol has the same policy shape. The wallet first gives `GPv2VaultRelayer` an ERC-20 allowance through a wallet action. That approval can require owner and executor approval. Later, a registered signer signs a CoW order digest. A live allowlisted solver calls `GPv2Settlement.settle`, the settlement contract asks the wallet whether the signature is valid through ERC-1271 and the wallet returns the magic value because the signer is registered. The CoW VaultRelayer then pulls the sell token from the wallet and the settlement pays the signed receiver. The executor is not asked to approve that specific CoW order, receiver, appData, partial-fill amount, expiry, or fee.

This is normal CoW settlement behavior and CoW itself enforces its signed order fields. The wallet-side issue is the same as Permit2: if the product expects the executor to review every fund-moving external protocol spend, a live ERC-1271 owner signature plus a pre-existing approval is enough to bypass that executor review.

**— RECOMMENDATION**

Decide whether live ERC-1271 signatures are allowed to authorize fund movement without the executor. If they are not, do not accept arbitrary live ERC-1271 hashes for Permit2-style spend protocols, CoW-style settlement protocols, or other approved external spenders. Require a wallet pre-authorization created through an owner-and-executor signed transaction or add a wallet policy layer that only returns the ERC-1271 magic value for hashes that were registered with the intended protocol, token, amount, spender, recipient, deadline and order metadata.

For Permit2 specifically, prefer `permitWitnessTransferFrom` with a witness that binds the user-facing recipient and order details, but only through a spender/integration that enforces those witness fields against the actual transfer details. Avoid granting Permit2 reusable ERC-20 allowances unless the wallet also tracks and clears the matching Permit2 sub-allowances. Consider adding explicit wallet functions for Permit2 lockdown and nonce invalidation if Permit2 remains a supported shared spender.

**— RESOLUTION**

Status: Resolved.

MEDIUM **M-09**

RESOLVED

# Emergency blocks Permit2 cleanup and CoW cancel

CATEGORY Unexpected Behavior  
LOCATION src/facets/ExecutorExternalCallFacet.sol  
REVIEW Main Review

## DESCRIPTION

`executorExternalCall` is the wallet's generic way to make an external protocol see `msg.sender == wallet`. It always calls `LibDirectMode.enforceExecutorNotBlocked()`, which reverts during global pause and direct mode.

This is unsafe for external protocols whose cleanup or cancellation functions are owner-scoped to the caller. Permit2 cleanup functions such as `lockdown`, `invalidateNonces` and `invalidateUnorderedNonces` use `msg.sender` as the Permit2 owner. An outsider can call them, but that only changes the outsider's Permit2 state. To clean the wallet's Permit2 sub-allowances or nonces, the call must originate from the wallet.

CoW Protocol has the same cancellation requirement. `GPv2Settlement.invalidateOrder` must be called by the order owner. For this smart-contract wallet, that means the invalidation call must come from the wallet. An outsider cannot invalidate the wallet's CoW order.

During global pause, wallet-originated Permit2 `lockdown` and CoW `invalidateOrder` calls through `executorExternalCall` revert with `GlobalPauseActive`. During direct mode, wallet-originated Permit2 nonce invalidation and CoW `invalidateOrder` calls revert with `DirectModeActive`. Existing external protocol authorizations can still be consumed while these cleanup and cancellation calls are blocked. The fork tests show a Permit2 sub-allowance spending during global pause, a live ERC-1271 Permit2 transfer spending during direct mode after nonce invalidation was blocked and CoW settlement through a live allowlisted solver during both emergency states after order invalidation was blocked.

Consequently, emergency states do not provide a user-protective way to revoke, invalidate or cancel those authorizations when they most need to be cleaned up.

## RECOMMENDATION

Add protocol-specific emergency cleanup and cancellation functions instead of routing these calls through `executorExternalCall`. They should be callable during normal operation, global pause and direct mode after a valid owner authorization. Use dedicated EIP-712 type

**M-09** Emergency blocks Permit2 cleanup and CoW cancel

CONTINUED

hashes, signer-bound nonces, deadlines and the existing reentrancy guard. Do not call `enforceExecutorNotBlocked()` or `enforceUserNotBlocked()` from these functions, because they are user-protective revocations rather than executor operations.

At minimum, add wrappers for Permit2 `lockdown`, Permit2 ordered nonce invalidation, Permit2 unordered nonce invalidation and CoW `invalidateOrder`. The Permit2 wrappers should call the configured Permit2 address with zero ETH. The CoW wrapper should call the configured CoW settlement address with zero ETH and reject order UIDs that do not encode `address(this)` as the order owner. If the product also wants immediate approval shutdown, add an owner-authorized emergency zero-approval function for known shared spenders such as Permit2 and `GPv2VaultRelayer`, with checks that avoid accidentally clearing a newer tracked approval unless the owner explicitly authorizes that exact `(token, spender)`.

Avoid a broadly generic emergency external call as the primary fix. If a generic mechanism is still added, restrict it to fixed protocol addresses, fixed non-fund-moving selectors, zero ETH value and calldata shape checks for each selector. A protocol-specific wrapper is safer and easier to audit.

Finally, align live ERC-1271 validation with the emergency policy. During global pause or direct mode, live signer signatures should not keep authorizing new Permit2 spends or CoW settlements while the wallet is relying on emergency cleanup. Either return invalid for live ERC-1271 signatures from known spend protocols during those states or only return the magic value for hashes that were explicitly registered through a wallet function that remains revocable in the same state.

**— RESOLUTION**

Status: Resolved.

MEDIUM **M-10**

ACKNOWLEDGED

# Recovery-admin backdoor survives signer removal

CATEGORY Access Control  
LOCATION src/facets/RecoveryFacet.sol  
REVIEW Main Review

## — DESCRIPTION

`setRecoveryConfig` sets per-wallet recovery admins with a single owner signature. There is no executor co-signature, no timelock, and it takes effect immediately. `validateAdmins` only rejects the zero address, duplicates, and the current factory executor, so any attacker-controlled address is accepted as a recovery admin.

Separately, signer removal ( `executeRemoveOwner` and `executeRemovePasskey` ) only mutates the active signer arrays. It never touches recovery storage, so `customRecoveryAdmins` and `hasCustomConfig` persist across the removal.

Together these let a signer that is compromised for even a short time install a permanent recovery-admin backdoor. While the attacker holds a registered signer, it calls `setRecoveryConfig([attacker])`. This single call also replaces the wallet's existing recovery admins.

Removing the compromised signer no longer contains the compromise because the attacker stays a recovery admin and can silently call `initiateRecovery` then `executeRecovery` to add itself back and drain the wallet, while also denying the real user recovery since the legitimate admins were replaced.

## — RECOMMENDATION

Treat a signer-set change as a revocation event for recovery state. On owner or passkey removal, reset custom recovery configuration to factory defaults, or require explicit re-confirmation of recovery admins after the signer set changes. Make `setRecoveryConfig` safer to begin with: require an executor co-signature, or a timelock plus a cancellation window, so a single momentarily-compromised signer cannot silently install a permanent recovery-admin backdoor.

---

**M-10** Recovery-admin backdoor survives signer removal

CONTINUED

---

— **RESOLUTION**

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

# BatchExecutor cannot be factory executor

|          |                                  |
|----------|----------------------------------|
| CATEGORY | Unexpected Behavior              |
| LOCATION | script/DeployBatchExecutor.s.sol |
| REVIEW   | Main Review                      |

## DESCRIPTION

The deployment scripts and runbook direct operators to rotate `factory.executor()` to the deployed `BatchExecutor`. `DeployBatchExecutor.s.sol` lists `factory.scheduleExecutorChange(batchExecutor)` as the migration step and says to verify that `factory.executor() == batchExecutor`. The deployment runbook then calls `ScheduleExecutorChange.s.sol` with `NEW_EXECUTOR=$BATCH_EXECUTOR`.

That configuration is incompatible with wallet signature verification. Wallets call `LibAuth.verifyExecutorSignature`, which checks the signature against the current `factory.executor()` through `SignatureChecker.isValidSignatureNowCalldata`. This works for an EOA executor and for a contract executor that implements ERC-1271.

`BatchExecutor` does not implement ERC-1271 `isValidSignature`. Therefore, once the factory executor is set to `BatchExecutor`, the wallet tries to validate executor signatures by staticcalling `isValidSignature` on `BatchExecutor` and the check fails. Executor-signed operations such as allowance spends and dual-signature withdrawals then revert with `InvalidExecutorSignature` until governance rotates the executor back to a valid signer or to an ERC-1271-capable contract.

## RECOMMENDATION

Keep `factory.executor()` set to the backend signer or HSM-controlled EOA that actually produces executor signatures. Use `BatchExecutor` only as a relay that submits pre-signed wallet calls. If the executor must be a contract, implement ERC-1271 signature validation in that contract and update the deployment scripts and docs so they distinguish the signing executor from the batch relay.

## RESOLUTION

Status: Acknowledged.

LOW **L-02**

RESOLVED

# Permissionless nonce invalidation censors spends

|          |                               |
|----------|-------------------------------|
| CATEGORY | DoS                           |
| LOCATION | src/facets/AllowanceFacet.sol |
| REVIEW   | Main Review                   |

## — DESCRIPTION

`invalidateSpendNonces` is callable by anyone. It writes directly to the wallet's spend nonce bitmap and `spendDailyAllowance` rejects any spend whose nonce bit is already set.

This means a third party can cancel a signed daily-allowance spend if the nonce is predictable or exposed before execution. The caller does not need an owner signature, executor signature or privileged role. The effect is censorship, not theft: funds are not redirected, but the intended executor-signed settlement fails with `SpendNonceAlreadyUsed`.

The protocol documentation treats permissionless invalidation as an accepted design, provided the backend monitors `SpendNoncesInvalidated`, uses high-entropy nonce ranges and rotates nonces when a spend fails. The issue remains worth tracking because those safeguards are operational commitments rather than code-enforced guarantees.

## — RECOMMENDATION

Require owner or executor authorization for nonce invalidation or replace it with a signed cancellation message. If permissionless invalidation is kept, enforce high-entropy nonce generation in the backend, avoid exposing signed spend calldata before submission and treat `SpendNonceAlreadyUsed` as a mandatory nonce-rotation signal.

## — RESOLUTION

Status: Resolved.

LOW **L-03**

ACKNOWLEDGED

# Reused orderHash can strand old allowances

|          |                                                        |
|----------|--------------------------------------------------------|
| CATEGORY | Unexpected Behavior                                    |
| LOCATION | src/facets/ExternalApproveAndPreAuthorizeHashFacet.sol |
| REVIEW   | Main Review                                            |

## DESCRIPTION

`externalApproveAndPreAuthorizeHash` does not reject an `orderHash` that is already authorized or already has a deadline-gated approval entry. Reusing the same hash for a different `(token, spender)` pair overwrites the hash-keyed approval entry while leaving the first pair's `currentApprovalOwner` slot pointing at the reused hash.

When `clearStaleApproval` or `revokePreAuthorizedHash` later runs, it only sees the most recent entry. It can clear the second token approval and revoke the hash, but it no longer has the first token and spender needed to zero the original allowance. The original ERC-20 approval remains spendable even though the wallet has no usable cleanup entry for it.

For example, the wallet first uses hash `H` to approve `USDe` for `spenderA` and the cleanup entry for `H` points to `USDe` and `spenderA`. Later, the wallet reuses `H` to approve `tokenB` for `spenderB`. The cleanup entry for `H` now points only to `tokenB` and `spenderB`. When cleanup runs for `H`, it clears the second approval, but the old `USDe` approval for `spenderA` remains live. `spenderA` can still call `transferFrom` against the wallet.

This depends on a reused order hash, which should be rare if upstream protocols generate unique hashes for unique orders. The wallet still treats uniqueness as an off-chain assumption rather than an on-chain invariant.

## RECOMMENDATION

Reject reused hashes before approving the token or writing new metadata. A minimal fix is to revert when `LibPreAuthorizedHash.isAuthorized(orderHash)` or `LibDeadlineGatedApproval.exists(orderHash)` is already true. Alternatively, key cleanup entries by a wallet-generated authorization id that includes the order hash, token, spender, amount, nonce and approval deadline.

## RESOLUTION

Status: Acknowledged.

LOW **L-04**

RESOLVED

# Old orders use newer same-spender allowance

|          |                                                        |
|----------|--------------------------------------------------------|
| CATEGORY | Validation                                             |
| LOCATION | src/facets/ExternalApproveAndPreAuthorizeHashFacet.sol |
| REVIEW   | Main Review                                            |

## DESCRIPTION

`externalApproveAndPreAuthorizeHash` treats the ERC-20 allowance as one shared slot per `(token, spender)`, but it treats pre-authorized hashes as independent approvals. When a user signs a second order for the same token and spender, the function overwrites the token allowance and records the new `orderHash` as the current owner of that allowance. It does not revoke the old `orderHash`.

Consequently, an older order can remain ERC-1271-valid after a newer authorization funds the same spender. If the external protocol still accepts the older order, the spender can consume the newer allowance even though that allowance was created for a different order.

For example, a user signs Order A for `100 USDe` through a Fusion+ router. Before Order A is filled or explicitly cancelled, the backend creates Order B for `500 USDe` through the same router and calls `externalApproveAndPreAuthorizeHash` again. The wallet now gives the router a `500 USDe` allowance, but Order A is still ERC-1271-valid because its hash was never revoked. If the router or resolver can still fill Order A, it can pull tokens from the allowance intended for Order B. The user can therefore execute an older trade they no longer expected to be funded and the newer order's allowance can be partially consumed before Order B is filled.

The contract comments state that overlapping orders for the same `(token, spender)` pair are unsupported and must be avoided by the backend, but the contract does not enforce that invariant.

## RECOMMENDATION

When a new approval is recorded for a `(token, spender)` pair, revoke or reject the active hash that currently owns that pair. A safe minimal fix is to read the previous owner before `setCurrentOwner`, revoke the previous pre-authorized hash when it is nonzero and different from the new hash and clear its approval entry. Alternatively, revert when a current owner already exists and require explicit revocation before replacement.

---

**L-04** Old orders use newer same-spender allowance

CONTINUED

---

— **RESOLUTION**

Status: Resolved.

LOW **L-05**

PARTIALLY RESOLVED

## Pending signer ready views ignore expiry

CATEGORY Validation  
LOCATION `src/libraries/LibAuth.sol`  
REVIEW Main Review

### — DESCRIPTION

Pending signer additions and removals store both a cooldown timestamp and an expiry timestamp, but the readiness helpers only check whether the cooldown has elapsed. They do not check `expiresAt` and the public pending-action getters do not return the expiry timestamp.

As a result, `isPendingOwnerReady` and `isPendingOwnerRemovalReady` can report an expired action as ready even though execution will revert with `PendingActionExpired`. The passkey readiness helpers mirror the same pattern. This can mislead wallets or operators into presenting expired signer actions as executable until another transaction prunes them.

### — RECOMMENDATION

Include expiry in the readiness checks and expose it through the pending-action getters. `isPending*Ready` should return false when `block.timestamp > expiresAt`. UIs should also display or use `expiresAt` directly instead of inferring readiness from cooldown alone.

### — RESOLUTION

Status: Partially resolved.

LOW **L-06**

RESOLVED

## Direct mode cannot recover received NFTs

|          |                                          |
|----------|------------------------------------------|
| CATEGORY | Unexpected Behavior                      |
| LOCATION | src/facets/ExecutorExternalCallFacet.sol |
| REVIEW   | Main Review                              |

### DESCRIPTION

The wallet advertises ERC-721 and ERC-1155 receiver support and accepts those assets. Direct mode, however, only has direct ETH and ERC-20 withdrawal functions. Moving an NFT out of the wallet requires the wallet to call the NFT contract as `msg.sender`, which is only possible through `executorExternalCall`. `executorExternalCall` is blocked when direct mode is active. Consequently, a user who enters direct mode because the executor is unavailable or untrusted can withdraw ETH and ERC-20 balances, but cannot recover NFTs held by the wallet without leaving direct mode and using the executor-gated generic call function.

### RECOMMENDATION

Add direct-mode asset recovery functions for token standards the wallet can receive. A narrow fix is to add direct ERC-721 and ERC-1155 transfer functions authorized by an owner signature and gated by active direct mode.

If the product wants broader user sovereignty in direct mode, add a `directExternalCall` function with owner-only authorization, nonce/deadline protection and careful target/value restrictions.

### RESOLUTION

Status: Resolved.

LOW **L-07**

PARTIALLY RESOLVED

# Executor calls create untracked approvals

|          |                                          |
|----------|------------------------------------------|
| CATEGORY | Trust Assumptions                        |
| LOCATION | src/facets/ExecutorExternalCallFacet.sol |
| REVIEW   | Main Review                              |

## DESCRIPTION

`executorExternalCall` allows an owner and executor to authorize arbitrary calldata to any nonzero, non-self target. That includes direct ERC-20 `approve` calls from the wallet and approval-style calls into shared spenders such as Permit2. These approvals bypass the deadline-gated approval registry used by the dedicated approval and pre-authorization facet.

The resulting allowance is real at the token contract, but `currentApprovalOwner` and the stale approval cleanup paths do not know about it because no registry entry was created.

This can also corrupt later cleanup for the same `(token, spender)` slot. For example, a tracked approval can expire while `currentApprovalOwner` still points at its hash. If a later generic `approve` call replaces the real ERC-20 allowance without updating the registry, `clearStaleApproval` will still believe the expired hash owns the slot and will zero the newer untracked allowance.

Permit2 adds a second version of the same problem. `Permit2.approve(token, spender, amount, expiration)` stores a Permit2 sub-allowance keyed by `(wallet, token, spender)`. Clearing the wallet's ERC-20 approval to Permit2 does not clear that sub-allowance. If the wallet later grants Permit2 a fresh ERC-20 allowance for another operation, the old Permit2 spender can become funded again until its Permit2 expiration passes.

**L-07** Executor calls create untracked approvals

CONTINUED

The missing registry entry has a further consequence once the wallet enters direct mode: an untracked approval cannot be revoked at all. Every function that can lower an allowance ( `executorExternalCall` , `executorApproveAndExternalCall` , `externalApprove` ) begins with `enforceExecutorNotBlocked()` and reverts while direct mode is active. The two revoke paths that remain available in direct mode, `revokePreAuthorizedHash` and `clearStaleApproval` , only zero an allowance that has a registry entry, which an `executorExternalCall` approval never created. So a user who enters direct mode because they no longer trust the executor can sweep the wallet's current balance but cannot cut off a standing approval the executor left to a colluding spender. Since this is a payment account that keeps receiving tokens, every inbound transfer stays pullable by that approval for as long as the user remains in direct mode; the only way to revoke is to leave direct mode and route `approve(spender, 0)` back through the executor-cosigned path, re-involving the exact party the user was escaping.

**— RECOMMENDATION**

Block ERC-20 allowance-mutating selectors and known shared-spender approval selectors in `executorExternalCall` or route all approval-style calls through the deadline-gated approval facet. At minimum, the blocked selectors should include `approve(address,uint256)` , `increaseAllowance(address,uint256)` , and `Permit2.approve(address,address,uint160,uint48)` . If the implementation uses a selector denylist, also consider legacy or allowance-reducing variants such as `increaseApproval(address,uint256)` , `decreaseAllowance(address,uint256)` and `decreaseApproval(address,uint256)` so generic calls cannot desynchronize real token allowances from the wallet's registry.

If generic approvals remain supported, they should also clear or replace stale ownership metadata for the same `(token, spender)` slot so old tracked approvals cannot later zero newer untracked allowances.

For Permit2, cleanup should also call `Permit2.approve(token, spender, 0, 0)` or `Permit2.lockdown` for the exact `(token, spender)` sub-allowance created by the wallet operation.

**— RESOLUTION**

Status: Partially resolved.

LOW **L-08**

RESOLVED

# Empty recovery reset keeps opt-out active

CATEGORY Unexpected Behavior  
LOCATION src/facets/RecoveryFacet.sol  
REVIEW Main Review

## DESCRIPTION

`setRecoveryConfig([])` is treated as a reset to factory default recovery admins. In the empty-array branch, the contract deletes `customRecoveryAdmins` and sets `hasCustomConfig = false`, but it does not clear `rs.optedOut`.

`LibRecovery.getEffectiveAdmins` checks `optedOut` before it checks `hasCustomConfig`. If `optedOut` is still true, it returns an empty admin list and never reads the factory defaults.

For example, a wallet owner can opt out of recovery while traveling. Later, the UI may offer a "restore default recovery" action that calls `setRecoveryConfig([])`. The transaction succeeds and the wallet no longer has a custom config, but recovery remains disabled because `optedOut` is still true. The owner can believe factory recovery admins are active while the wallet has no effective recovery admins.

This can leave recovery unintentionally disabled and increase account lockout risk.

## RECOMMENDATION

Set `rs.optedOut = false` in the `admins.length == 0` branch before returning. If empty arrays should not re-enable recovery, expose a separate explicit function for restoring factory defaults and make the UI/API distinction clear.

## RESOLUTION

Status: Resolved.

LOW **L-09**

RESOLVED

# BatchExecutor owner can freeze config

|          |                       |
|----------|-----------------------|
| CATEGORY | Unexpected Behavior   |
| LOCATION | src/BatchExecutor.sol |
| REVIEW   | Main Review           |

## DESCRIPTION

`BatchExecutor` inherits `OpenZeppelin Ownable2Step` but does not override `renounceOwnership`. The factory disables ownership renunciation, but the batch executor does not mirror that protection.

If the owner renounces ownership, no address can add or remove authorized callers, update approved or blocked recipients or pause and unpause the batch executor. If renunciation happens while the contract is paused, batch execution remains permanently paused. If it happens while unpaused, stale authorized callers and recipient policy remain fixed forever.

This requires an owner action, so it is an operational issue rather than an unprivileged exploit. It is still a concrete configuration-lock risk for a security-sensitive relay component.

## RECOMMENDATION

Disable ownership renunciation on `BatchExecutor`, matching the factory.

```
error RenounceOwnershipDisabled();  
function renounceOwnership() public pure override {  
    revert RenounceOwnershipDisabled();  
}
```

## RESOLUTION

Status: Resolved.

LOW **L-10**

RESOLVED

# Upgrade script assumes EOA ProxyAdmin owner

|          |                             |
|----------|-----------------------------|
| CATEGORY | Configuration               |
| LOCATION | script/UpgradeFactory.s.sol |
| REVIEW   | Main Review                 |

## DESCRIPTION

`UpgradeFactory.s.sol` is written for a `ProxyAdmin` controlled directly by an EOA private key. It reads `PRIVATE_KEY`, checks that `admin.owner()` equals `vm.addr(ownerKey)` and then broadcasts `ProxyAdmin.upgradeAndCall` with that key.

Production ownership is intended to sit behind a `TimelockController`. After the `ProxyAdmin` owner is transferred to the timelock, no private key can satisfy this script's pre-flight check or directly broadcast the upgrade call.

Consequently, the production upgrade runbook and the production ownership model do not match. Operators may discover during an upgrade that the provided script cannot execute the intended timelock-owned upgrade or may be tempted to keep or restore EOA ownership just to use the script.

## RECOMMENDATION

Replace the production factory upgrade script with a timelock-aware workflow. The script should deploy or reference the new implementation, prepare the `ProxyAdmin.upgradeAndCall` calldata, schedule it through `TimelockController` and provide a separate execution step after the delay.

Keep any direct-EOA upgrade script clearly marked for local or staging deployments only. Production scripts and docs should not require `PRIVATE_KEY` to be the `ProxyAdmin` owner once ownership has moved to the timelock.

## RESOLUTION

Status: Resolved.

LOW **L-11**

PARTIALLY RESOLVED

# Batch policy misses executor withdrawals

CATEGORY Validation  
LOCATION src/BatchExecutor.sol  
REVIEW Main Review

## DESCRIPTION

`BatchExecutor.batchExecute` only applies `approvedRecipients` and `blocked` checks when calldata targets `spendDailyAllowance` or `spendDailyAllowanceEth`. The same batch function accepts arbitrary wallet calldata, including `executorEthWithdrawal` and `executorTokenWithdrawal`, but those selectors are not parsed for recipient policy.

Consequently, an authorized batch caller can relay a valid owner-and-executor-signed withdrawal to a recipient that the `BatchExecutor` blocklist would reject for daily allowance spends. The withdrawal still requires valid wallet signatures, so this is not an unauthenticated drain. It does mean the `BatchExecutor` recipient policy is not a universal compliance control for batched value transfers.

## RECOMMENDATION

Either restrict `batchExecute` to the supported allowance-spend selectors or extend recipient extraction and policy checks to every batched wallet function that can transfer value or grant spending rights. At minimum, document that `approvedRecipients` and `blocked` only protect daily allowance settlement calls.

## RESOLUTION

Status: Partially resolved.

LOW **L-12**

PARTIALLY RESOLVED

# Dual-sign ops are not bound to executor

CATEGORY Validation  
LOCATION src/libraries/LibAuth.sol  
REVIEW Main Review

## DESCRIPTION

`verifyExecutorSignature` checks `executorSig` against whichever address `factory.executor()` returns when the transaction executes. The owner-signed EIP-712 digests do not name the executor and do not include a factory executor epoch.

This means an owner can sign a dual-signature operation while executor E1 is active, but the operation can later be completed by executor E2 after the factory rotates executors. The owner nonce must still be unused and the signature deadline must still be valid. E2 cannot change the signed operation parameters, but it can provide the executor half of an authorization the owner created under a different executor.

The issue is broader than allowance updates. The same verification model is used by `instantSetAllowance`, `externalApprove`, `externalApproveAndPreAuthorizeHash`, executor withdrawals, `executorExternalCall` and `executorApproveAndExternalCall`.

This is not an unsigned execution issue: the wallet still requires a valid owner signature and a valid signature from the executor that is current at execution time. The risk is that a new executor can complete old owner-signed operations after rotation, even though the owner never signed a message that named that new executor.

## RECOMMENDATION

Bind each dual-signature EIP-712 struct to the executor that is expected to co-sign or bind it to a factory-level `executorEpoch`. If using an epoch, store it in the factory, increment it in `executeExecutorChange`, include it in every owner-and-executor digest and require the signed epoch to match the current epoch before consuming the owner nonce.

Apply this to `InstantSetAllowance`, `ExternalApprove`, `ExternalApproveAndPreAuthorizeHash`, `ExecutorWithdrawal`, `ExecutorExternalCall` and `ExecutorApproveAndExternalCall`. If outstanding authorizations should survive executor rotation, document that behavior and keep deadlines short enough to limit stale owner signatures.

---

**L-12** Dual-sign ops are not bound to executor

CONTINUED

---

— **RESOLUTION**

Status: Partially resolved.

LOW **L-13**

RESOLVED

# Scheduled governance actions never expire

|          |                                |
|----------|--------------------------------|
| CATEGORY | Unexpected Behavior            |
| LOCATION | src/UserProxyFactoryBeacon.sol |
| REVIEW   | Main Review                    |

## DESCRIPTION

The factory's timelocked governance actions enforce a minimum delay but no maximum. Each execute function checks only that `block.timestamp` is at least `scheduledAt + UPGRADE_DELAY`, with no upper bound:

```
function executeUpgrade() external onlyOwner {
  if (pendingImplementation == address(0)) revert NoUpgradeScheduled();
  if (block.timestamp < upgradeScheduledAt + UPGRADE_DELAY) revert UpgradeNotReady();
  // ... no upper-bound / expiry check ...
}
```

The same pattern is used by:

- `executeUpgrade` → `block.timestamp >= upgradeScheduledAt + UPGRADE_DELAY`
- `executeExecutorChange` → `block.timestamp >= executorChangeScheduledAt + UPGRADE_DELAY`
- `executeDefaultRecoveryAdmins` → `block.timestamp >= recoveryConfigScheduledAt + UPGRADE_DELAY`
- `executeFacetUpdate` → `block.timestamp >= facetUpdateScheduledAt + UPGRADE_DELAY`

Once the 48-hour delay has elapsed, a pending action remains executable indefinitely. If governance schedules an action, then reconsiders but does not call the matching cancel function, the stale action stays live and can be executed at any later point.

A later execution can silently apply a decision that no longer reflects current intent. For example, a beacon upgrade scheduled months earlier could be executed long after a newer implementation has superseded it; a scheduled executor change could rotate to a key that has since been retired; or a stale default-recovery-admin set could be installed after the intended roster changed. Because there is no expiry, the only safeguard against a forgotten schedule is an explicit cancel, which depends on operators remembering the pending action exists.

---

**L-13** Scheduled governance actions never expireCONTINUED

---

**— RECOMMENDATION**

Add an expiry window to scheduled governance actions. After `scheduledAt + UPGRADE_DELAY` plus a bounded grace period, require the action to be re-scheduled rather than executed or consider documenting this behavior.

**— RESOLUTION**

Status: Resolved.

LOW **L-14**

RESOLVED

# Executor swap can brick a pre-auth order

CATEGORY Unexpected Behavior  
LOCATION src/facets/ExecutorExternalCallFacet.sol  
REVIEW Main Review

## DESCRIPTION

`externalApproveAndPreAuthorizeHash` records a pre-authorized order by setting the ERC-20 allowance for `(token, spender)`, marking the order hash authorized in `LibPreAuthorizedHash`, writing a deadline-gated entry, and setting `currentOwnerHash[token][spender]` to that order hash. From then on, `ERC1271Facet.isValidSignature` returns the magic value for the order hash, and a resolver can fill the order by pulling tokens against the live allowance.

`executorApproveAndExternalCall` performs the following, and never reads or updates the deadline-gated approval registry or the pre-authorized hash storage:

```
IERC20(token).forceApprove(spender, amount); // overwrites the order's allowance
(success, result) = target.call{value: value}(data);
if (!success) revert ExternalCallFailed(result);
IERC20(token).forceApprove(spender, 0); // zeroes it after the call
```

If an executor swap is routed through `executorApproveAndExternalCall` for the same `(token, spender)` pair that a pending pre-authorized order already uses, the first `forceApprove` overwrites the order's allowance and the trailing `forceApprove(spender, 0)` zeroes it. The registry is never updated: the order hash is still authorized, `currentOwnerHash[token][spender]` still points at it, and `isValidSignature` still returns the magic value. On-chain, however, the allowance is now zero.

The order is therefore bricked while still appearing valid. A resolver that queries `isValidSignature` gets a positive result and attempts the fill, but the `transferFrom` reverts because the allowance was zeroed by the unrelated swap. The user's standing order silently stops being fillable until it is re-approved or expires.

## RECOMMENDATION

Coordinate `executorApproveAndExternalCall` with the deadline-gated approval registry. Before overwriting an allowance for a `(token, spender)` pair, check whether a tracked order currently owns that slot and either reject the call, revoke the affected pre-authorized

**L-14** Executor swap can brick a pre-auth order

CONTINUED

hash and clear its entry, or route all approval-style flows through the deadline-gated approval facet so registry state and on-chain allowance cannot diverge. At minimum, document that `executorApproveAndExternalCall` must not target a `(token, spender)` pair with an active pre-authorized order.

**— RESOLUTION**

Status: Resolved.

LOW **L-15**

PARTIALLY RESOLVED

# Gas guard can still revert batches

|          |                       |
|----------|-----------------------|
| CATEGORY | Unexpected Behavior   |
| LOCATION | src/BatchExecutor.sol |
| REVIEW   | Main Review           |

## DESCRIPTION

`BatchExecutor.batchExecute` tries to stop gracefully when gas is low by checking `gasleft() <= 5000` before each item. That check only looks at gas before the next external call. It does not reserve gas for the work `batchExecute` must still do after that call returns.

The low-level `targets[i].call(calldatas[i])` forwards almost all remaining gas to the target. If the target consumes that gas, `BatchExecutor` gets back only the EIP-150 reserve. That reserve may be too small to finish post-call accounting, emit `BatchExecuteCompleted`, ABI-encode the returned `bool[]` and run the `nonReentrant` cleanup. In that case the transaction can still run out of gas and revert the whole batch, rolling back earlier successful items.

For example:

- A relayer submits a card-settlement batch with many normal wallet spends and one unexpectedly gas-heavy wallet call near the end.
- The early spends succeed and transfer funds to merchants.
- The gas-heavy item consumes almost all forwarded gas.
- When control returns to `BatchExecutor`, there is not enough gas left to record the final result, emit the completion event, return the results array and reset the reentrancy guard.
- The whole transaction reverts, so the earlier merchant transfers are rolled back even though the batch logic is intended to isolate failed items or truncate before gas exhaustion.

The caller can mitigate this by supplying enough gas and avoiding unexpectedly expensive targets, so this is simply an operational reliability issue. It still matters because the contract explicitly tries to prevent low-gas batch reverts, but the current guard does not reserve enough gas to make that guarantee reliable.

## RECOMMENDATION

**L-15** Gas guard can still revert batches

CONTINUED

Reserve enough gas for post-call accounting and completion before forwarding gas to each target. For example, require `gasLeft()` to exceed a conservative completion reserve, then call each target with `gas: gasLeft() - reserve`. If graceful truncation is not required, remove the low-gas truncation logic and document that the relayer must provide enough gas for the full batch.

**— RESOLUTION**

Status: Partially resolved.

LOW **L-16**

RESOLVED

# Executor rotation can overlap wallet roles

|          |                                |
|----------|--------------------------------|
| CATEGORY | Validation                     |
| LOCATION | src/UserProxyFactoryBeacon.sol |
| REVIEW   | Main Review                    |

## DESCRIPTION

Wallet functions prevent the current factory executor from being added to privileged wallet roles.

`initiateAddOwner` and `executeAddOwner` reject owner additions when the pending owner is the current executor. `setRecoveryConfig` rejects recovery admins that equal the current executor. Those checks only run when a wallet role is configured. `executeExecutorChange` does not perform the inverse check when the executor changes. After the timelock, it assigns `executor = pendingExecutor` without checking whether the new executor is already an owner or recovery admin for existing wallets.

If the executor is rotated to an address that already owns a wallet, the same key can satisfy both the owner signature and the executor signature for that wallet. This breaks the intended separation between wallet-owner authorization and executor co-signing.

If the executor is rotated to an address that is already a recovery admin, the same address can initiate recovery for that wallet. This breaks the intended separation between executor co-signing authority and recovery authority.

This requires an executor rotation by the factory owner. The issue still matters because that rotation can silently invalidate role-separation assumptions that existing wallets already relied on.

## RECOMMENDATION

Treat executor rotation as a role-conflict change. Reject scheduled executor changes when the new executor is known to already be a wallet owner or recovery admin.

Because the factory may not know every per-wallet role assignment, also enforce the separation where the roles are consumed. Dual-signature wallet operations should reject address-based owner signers that resolve to the current factory executor. `RecoveryFacet.initiateRecovery` should reject `msg.sender == factory.executor()`.

**L-16** Executor rotation can overlap wallet roles

CONTINUED

Also validate default recovery admin changes against the current executor and any pending executor. If feasible, maintain an indexed registry of wallet owners and recovery admins so the factory can block known conflicts before `executeExecutorChange` updates the executor.

**— RESOLUTION**

Status: Resolved.

LOW **L-17**

RESOLVED

## externalApprove silently bricks live orders

CATEGORY Logical Error  
LOCATION src/facets/ExternalApproveFacet.sol  
REVIEW Main Review

### — DESCRIPTION

A pre-authorized order and a plain token approval end up competing for the exact same on-chain state, and `externalApprove` resolves that conflict by silently clobbering whatever was there before.

When `externalApproveAndPreAuthorizeHash` sets up an order, it does three things for a given `(token, spender)`:

- sets the ERC-20 allowance via `forceApprove(spender, amount)`,
- flags the `orderHash` so that `isValidSignature(orderHash)` answers with the magic value,
- records that `orderHash` as the owner of the allowance slot (`setCurrentOwner(token, spender, orderHash)`).

A later call to `externalApprove` on the same pair sets its own allowance and stamps its own `approveHash` as the new owner, but it does not look at who owned the slot first, and it does not turn off the earlier order's pre-authorization flag. The earlier order therefore keeps reporting itself as a valid signature even though the allowance sitting behind it has been replaced.

`externalApprove` also never checks that the approved amount is nonzero; it only guards against a zero `token`, a zero `spender`, and out-of-range deadlines before forwarding the signed `amount` straight into `forceApprove`:

```
if (token == address(0)) revert ZeroAddress();  
if (spender == address(0)) revert ZeroAddress();  
if (approvalDeadline <= block.timestamp) revert InvalidApprovalDeadlinePast();  
if (approvalDeadline > type(uint64).max) revert InvalidApprovalDeadlineTooFar();  
// ... no 'amount != 0' check ...  
IERC20(token).forceApprove(spender, amount);
```

So a single approval call to a pair that already hosts a live order can either shrink, grow, or completely remove that order's funding while the order still looks fillable from the outside.

---

**L-17** externalApprove silently bricks live orders

---

CONTINUED

---

**RECOMMENDATION**

Before `externalApprove` overwrites the allowance for a `(token, spender)` pair, it should hand off the slot cleanly instead of silently replacing it.

**RESOLUTION**

Status: Resolved.

LOW **L-18**

RESOLVED

# Silent batch truncation is under-observable

CATEGORY Events  
LOCATION src/BatchExecutor.sol  
REVIEW Main Review

## DESCRIPTION

`batchExecute` tries to stop gracefully when gas runs low by breaking out of the loop before the next item:

```
for (uint256 i = 0; i < len;) {  
  if (gasleft() <= 5000) break; // silent early exit  
  attempted = i + 1;  
  ...  
  (bool success,) = targets[i].call(calldatas[i]);  
  ...  
}  
emit BatchExecuteCompleted(batchId, attempted, succeeded);
```

When the gas guard fires, the loop breaks and the function still completes "successfully." The only signal emitted is `BatchExecuteCompleted(batchId, attempted, succeeded)`, where `attempted = i + 1` is the number of items processed before the break, not the requested `targets.length`. The event does not carry the requested batch size at all.

## RECOMMENDATION

Make truncation observable on-chain. Emit the requested batch size ( `targets.length` ) alongside `attempted` in `BatchExecuteCompleted`, or emit a distinct event (e.g. `BatchTruncated(batchId, attempted, requested)` ).

## RESOLUTION

Status: Resolved.

LOW **L-19**

RESOLVED

# Cancel signature not bound to its recovery

CATEGORY Signatures  
LOCATION src/libraries/LibRecovery.sol  
REVIEW Main Review

## DESCRIPTION

A `cancelRecovery` signature should cancel one specific recovery. It doesn't, it can cancel a different one.

The signed message includes `pendingInitiatedAt` (the recovery's timestamp) but not `pendingNewSigner` (which recovery it is). So the only thing tying the signature to a recovery is its timestamp. Two recoveries created in the same block share the same timestamp, so a signature for the first one also works on the second.

This is reachable because `setRecoveryConfig` (and `optOutRecovery`) can cancel a pending recovery without using the cancel signer's nonce. So:

1. Owner A signs a cancel for recovery A but doesn't broadcast it.
2. Owner B calls `setRecoveryConfig`, which cancels recovery A. Owner A's nonce is untouched.
3. A recovery admin starts recovery B in the same block, it gets the same timestamp as A.
4. Owner A's old signature now cancels recovery B instead.

## RECOMMENDATION

Bind the recovery target into the cancel signature so it only works on the recovery it was meant for. Either:

- Add `pendingNewSigner` (hashed) to `CANCEL_RECOVERY_TYPEHASH` and into `hashCancelRecovery`, or
- Add a per-recovery id that increments on each `initiateRecovery` and bind it into the cancel signature.

Otherwise, acknowledge and document the behavior.

**PROOF OF CONCEPT** <https://github.com/GuardianOrg/ethenapay-wallet-contracts-source-fuzz-1781017310427/blob/fuzz-ali/test/fuzzing/handlers/FuzzRecovery.sol#L679>

---

**L-19** Cancel signature not bound to its recovery

CONTINUED

---

— **RESOLUTION**

Status: Resolved.

LOW **L-20**

ACKNOWLEDGED

# Pending passkey removal view returns stale index

CATEGORY Validation  
LOCATION src/libraries/LibAuth.sol  
REVIEW Main Review

## DESCRIPTION

`PendingPasskeyRemoval` stores a `passkeyIndex` that is captured when the removal is initiated in `initiateRemovePasskey` and is never updated afterward:

```
aus.pendingPasskeyRemovals[actionId] = PendingPasskeyRemoval({  
  passkeyIndex: index, // captured here, never refreshed  
  qx: _qx,  
  qy: _qy,  
  ...  
});
```

The active `passkeys` array is maintained with swap-and-pop, so removing any other passkey moves the last element into the removed slot and changes the indices of existing entries:

```
aus.passkeys[indexToRemove] = aus.passkeys[passkeyLen - 1];  
aus.passkeys.pop();
```

After such a reordering, the `passkeyIndex` recorded in a still-pending removal no longer points to the passkey it was created for, and may point to an unrelated passkey or be out of bounds.

`getPendingPasskeyRemoval` returns this stale index to off-chain callers, so any consumer that trusts `passkeyIndex` to identify the passkey scheduled for removal can act on the wrong entry, for example displaying or operating on a different passkey than the user intended to remove.

## RECOMMENDATION

Do not expose a cached array index that can drift. Either remove `passkeyIndex` from `PendingPasskeyRemoval` and the getter and rely solely on `qx` and `qy` for identity, or recompute the current index on read by scanning for the stored coordinates. If the index is kept for convenience, document that it is only valid at initiation time and must not be used after any other passkey removal.

---

**L-20** Pending passkey removal view returns stale index

CONTINUED

---

— **RESOLUTION**

Status: Acknowledged.

INFORMATIONAL **I-01**

PARTIALLY RESOLVED

# Default daily limits allow duplicate arrays

|          |                                |
|----------|--------------------------------|
| CATEGORY | Validation                     |
| LOCATION | src/UserProxyFactoryBeacon.sol |
| REVIEW   | Main Review                    |

## DESCRIPTION

`setDefaultDailyLimits` checks only that token and amount arrays have equal length and that each amount is below the maximum. It does not cap array length and does not reject duplicate token entries. New wallet initialization loops over the full configured array and applies each entry in order, so duplicate tokens are silently overwritten by later entries.

This is owner-controlled and only affects future wallet deployments, but it creates avoidable operational risk and gas griefing against new wallet creation.

## RECOMMENDATION

Add a conservative maximum length for default daily limit arrays and reject duplicate token addresses. Emit or store the canonical deduplicated configuration so off-chain tooling and wallet initialization agree on the effective defaults.

## RESOLUTION

Status: Partially resolved.

## INFORMATIONAL I-02

PARTIALLY RESOLVED

# WebAuthn RP ID hash is not enforced on-chain

|          |                           |
|----------|---------------------------|
| CATEGORY | Warning                   |
| LOCATION | src/libraries/LibAuth.sol |
| REVIEW   | Main Review               |

## DESCRIPTION

`LibAuth._verifyWebAuthn` accepts a WebAuthn assertion after `WebAuthn.verify` confirms the challenge, user presence, optional user verification and P-256 signature. The wallet does not compare the first 32 bytes of `authenticatorData` against an expected RP ID hash.

Consequently, the on-chain verifier relies on frontend and platform controls to keep passkey assertions scoped to the intended relying party. If those controls fail, an assertion signed by a registered passkey can still verify on-chain as long as the challenge matches. Nonces and deadlines still apply, so this is a domain-separation hardening issue rather than an unauthenticated signing bypass.

## RECOMMENDATION

Store or configure the expected RP ID hash and compare it to `bytes32(auth.authenticatorData[0:32])` before accepting a WebAuthn assertion. Keep frontend origin validation as defense in depth, but do not make it the only relying-party check for wallet authorization.

## RESOLUTION

Status: Partially resolved.

## INFORMATIONAL I-03

ACKNOWLEDGED

# Batch accepts non-wallet call targets

|          |                       |
|----------|-----------------------|
| CATEGORY | Warning               |
| LOCATION | src/BatchExecutor.sol |
| REVIEW   | Main Review           |

## DESCRIPTION

`BatchExecutor.batchExecute` is documented as executing calldata on wallet proxy targets, but it does not enforce that each `target` is a wallet proxy or even a deployed contract. After checking only that `msg.sender` is an authorized caller and that the arrays line up, it executes each item with `targets[i].call(calldatas[i])`.

This gives authorized callers the ability to make arbitrary calls from the `BatchExecutor` address. For example, if ERC-20 tokens are accidentally sent to the batch executor, an authorized caller can include the token contract as the batch target and call `transfer`, moving those tokens as `msg.sender == BatchExecutor`. The same missing target validation also means a call to an EOA or wrong address returns low-level success and is counted as a successful batch item even though no wallet operation executed.

## RECOMMENDATION

Validate batch targets before forwarding calls. If `BatchExecutor` is only meant to relay wallet operations, require each target to be a known wallet proxy from the factory or at minimum require `target.code.length > 0` and reject non-wallet targets. Also consider emitting per-item target and result data so off-chain settlement can detect invalid target submissions.

## RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-04

ACKNOWLEDGED

# Policy reverts abort entire batch

|          |                       |
|----------|-----------------------|
| CATEGORY | Warning               |
| LOCATION | src/BatchExecutor.sol |
| REVIEW   | Main Review           |

## DESCRIPTION

`BatchExecutor.batchExecute` states that one failed item should not abort the batch and the low-level wallet calls are recorded in the `results` bitmap. However, the recipient policy checks for `spendDailyAllowance` and `spendDailyAllowanceEth` happen before the low-level call and use direct `revert` statements when the recipient is blocked or not approved.

As a result, a single blocked or unapproved recipient aborts the entire transaction and rolls back earlier successful items. For example, a batch with a valid spend to an approved merchant followed by a spend to a newly blocked merchant reverts as soon as the blocked item is checked. The valid first spend is rolled back and no `BatchExecuteCompleted` event is emitted for the partial outcome.

## RECOMMENDATION

Keep recipient-policy failures inside the per-item result path. Instead of reverting on `RecipientBlocked` or `RecipientNotApproved`, mark that item as failed, continue processing the rest of the batch and emit enough per-item information for off-chain systems to identify the rejected recipient. If policy failures are intentionally meant to abort the whole batch, update the function documentation and monitoring expectations to make that behavior explicit.

## RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-05

ACKNOWLEDGED

# Recovery admins cannot replace pending recovery

|          |                              |
|----------|------------------------------|
| CATEGORY | Warning                      |
| LOCATION | src/facets/RecoveryFacet.sol |
| REVIEW   | Main Review                  |

## DESCRIPTION

`initiateRecovery` rejects every new recovery while `pendingActive` is true. The only cancellation function is `cancelRecovery`, and it requires a valid owner signature. Recovery admins therefore cannot cancel or replace a pending recovery, even when they are still current admins for the wallet.

This gives any recovery admin control over the single pending recovery slot once they call `initiateRecovery`. If that admin queues the wrong signer, whether by mistake or compromise, the rest of the recovery admin set cannot correct it. If the wallet owner cannot sign `cancelRecovery`, the wallet is left with two practical choices: execute the unintended pending recovery after the 10-day delay or leave recovery blocked.

For example, a user loses access to their only working signer and asks two configured recovery admins to help restore the wallet. The first admin enters the wrong address and initiates recovery to that signer. The second admin notices the problem before the delay expires and tries to start recovery to the correct replacement signer, but `initiateRecovery` reverts with `RecoveryAlreadyPending`. After the 10-day delay, the same restriction still prevents replacement. Anyone can instead execute the queued recovery and add the unintended signer as an owner.

This matters most when recovery is actually needed because the user has lost access to existing signers. In that situation the owner-signed cancellation control is unavailable, so a bad pending recovery can prevent the configured recovery admins from restoring the wallet to the intended signer.

## RECOMMENDATION

Keep owner-signed `cancelRecovery` as the highest-authority cancellation control, but add a recovery-admin fallback for cases where no owner signature is available. A safe design should store enough pending-recovery metadata to authorize correction, emit an explicit cancellation or replacement event, and prevent silent overwrites.

**I-05** Recovery admins cannot replace pending recovery

CONTINUED

For example, store the pending initiator and allow the initiating admin or a quorum of current recovery admins, to cancel or replace an active pending recovery. Also consider adding an expiry so a pending recovery cannot block future recovery forever when no owner signature is available.

**— RESOLUTION**

Status: Acknowledged.

## INFORMATIONAL I-06

ACKNOWLEDGED

# External approval docs omit approvalDeadline

CATEGORY Documentation  
LOCATION docs/API.md  
REVIEW Main Review

## DESCRIPTION

Several operator-facing docs describe `externalApprove` and `externalApproveAndPreAuthorizeHash` without the `approvalDeadline` field. The current contracts include `approvalDeadline` in both the external function ABI and the EIP-712 type hashes: `ExternalApproveFacet` hashes `ExternalApprove(bytes32 signer,address token,address spender,uint256 amount,uint256 approvalDeadline,uint256 nonce,uint256 deadline)` and `ExternalApproveAndPreAuthorizeHashFacet` likewise hashes `approvalDeadline` before `nonce` and `deadline`.

The stale ABI and typed-data descriptions appear in `API.md`, `SECURITY.md`, `MIGRATION.md` and `USER_GUIDE_VERIFY_BEFORE_SIGNING.md`. `MIGRATION.md` is especially misleading because it says the breaking typehash change was only the addition of `bytes32 signer`, while the deployed approval payloads also require the cleanup deadline.

The approval events are also inconsistent. `API.md` omits `approveHash` and `approvalDeadline` from `ExternalApproval` and `MONITORING.md` lists `ExternalApprovalAndHashAuthorization` with `approvalDeadline` before the hash. The contract event order is `token`, `spender`, `amount`, `orderHash`, `approvalDeadline`.

## RECOMMENDATION

Update every `ExternalApprove` and `ExternalApproveAndPreAuthorizeHash` example to include `approvalDeadline` in the same position as the contracts. The migration checklist should explicitly call out both breaking changes: the added `signer` field and the added approval cleanup deadline. Also update the event examples to match the compiled ABI, including `approveHash` for `ExternalApproval` and the correct `orderHash`, `approvalDeadline` order for `ExternalApprovalAndHashAuthorization`.

## RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-07

ACKNOWLEDGED

# Query guides use nonexistent view calls

CATEGORY Documentation  
LOCATION docs/OPERATOR\_QUERY\_GUIDE.md  
REVIEW Main Review

## DESCRIPTION

OPERATOR\_QUERY\_GUIDE.md says every `cast call` example uses the exact signature, but several examples target functions that are not in the current wallet or factory ABI. The guide tells operators to call `currentDayStart`, `isInDirectMode`, `directModeActivatesAt`, `exitDirectModeActivatesAt`, factory-level `getRecoveryConfig(address)`, `isRecoveryOptedOut`, `getActiveRecovery`, `pendingOwnerCount` and factory `isGloballyPaused`. USER\_GUIDE\_VERIFY\_BEFORE\_SIGNING.md repeats the stale direct-mode and global-pause calls and also queries recovery config on the factory.

The current contracts expose different views. `DirectModeFacet` exposes `isDirectMode()`, `pendingDirectMode()` and `pendingExitDirectMode()`. `UserProxyFactoryBeacon` exposes the public `globalPaused()` getter. `RecoveryFacet` exposes wallet-level `getRecoveryConfig()` and `getPendingRecoveryStatus()`. `AuthFacet` exposes `pendingOwnerAdditionCount()`, not `pendingOwnerCount()`. No public `currentDayStart()` view exists.

Operators following the docs therefore get selector-not-found reverts or failed factory calls when checking pause state, direct-mode state, recovery status, allowance reset state and pending signer operations. This weakens the intended pre-signing and operational safety checks because the documented checks cannot be executed as written.

## RECOMMENDATION

Replace the stale examples with the current ABI. Use `isDirectMode()(bool)`, `pendingDirectMode()(uint64,bool)`, `pendingExitDirectMode()(uint64,bool)`, `globalPaused()(bool)`, wallet-level `getRecoveryConfig()(address[],uint256,bool,bool)`, `getPendingRecoveryStatus()(bool,bytes,uint256,uint256)` and `pendingOwnerAdditionCount()(uint256)`. Remove `currentDayStart`.

## RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-08

ACKNOWLEDGED

# Runbook uses bytes16 wallet UUID selectors

|          |                    |
|----------|--------------------|
| CATEGORY | Documentation      |
| LOCATION | docs/DEPLOYMENT.md |
| REVIEW   | Main Review        |

## DESCRIPTION

The deployment runbook's wallet deployment examples call `predictWalletAddress(bytes16)` and `deployWalletWithPasskey(bytes16,bytes32,bytes32)`. The factory ABI uses `bytes32 walletUuid` for address prediction and wallet deployment.

Following the runbook literally produces calldata for selectors that the factory does not implement. Other docs already use `bytes32`, so this is an isolated but copy-pasteable ABI drift in the main deployment runbook.

## RECOMMENDATION

Change the deployment runbook examples to `predictWalletAddress(bytes32)(address)` and `deployWalletWithPasskey(bytes32,bytes32,bytes32)`. If backend UUIDs are naturally 16 bytes, document the exact expansion rule to the `bytes32 walletUuid` used as the CREATE2 salt before calling the factory.

## RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-09

ACKNOWLEDGED

# Batch docs use removed event name

|          |                                          |
|----------|------------------------------------------|
| CATEGORY | Documentation                            |
| LOCATION | docs/USER_GUIDE_VERIFY_BEFORE_SIGNING.md |
| REVIEW   | Main Review                              |

## DESCRIPTION

The user guide tells users to look up `BatchSpendExecuted(uint256,uint256,uint256)` after a batch and `API.md` documents the same removed event name. `PRODUCT.md` also says `BatchSpendExecuted` is emitted after batch settlement. The current `BatchExecutor` defines and emits `BatchExecuteCompleted(uint256 indexed batchId, uint256 total, uint256 succeeded)`.

A user, monitor or indexer built from these docs will not find batch execution logs.

## RECOMMENDATION

Replace `BatchSpendExecuted` with `BatchExecuteCompleted` in the user guide, API reference and product flow. Keep the monitoring guide as the source of truth for alerting fields and consider adding a small ABI-check test that validates documented event names against compiled ABIs.

## RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-10**

ACKNOWLEDGED

# Threat model overstates direct-mode caller risk

|          |                      |
|----------|----------------------|
| CATEGORY | Documentation        |
| LOCATION | docs/THREAT_MODEL.md |
| REVIEW   | Main Review          |

## — DESCRIPTION

The BatchExecutor selector-space analysis contradicts the current wallet authorization model. The direct-mode table correctly lists `initiateDirectMode`, `cancelDirectMode`, `initiateExitDirectMode` and `cancelExitDirectMode` as "Owner sig only". A later section then places three of those same functions under "Selectors a Compromised `authorizedCaller` Can Route Without Owner Co-Signature" and says the call will succeed through `batchExecute` subject only to direct-mode state preconditions.

The contract does not behave that way. Each direct-mode control function takes `signer`, `nonce`, `deadline` and `ownerSig`, then calls `LibAuth.verifyOwnerSignature`, checks that the recovered signer matches `signer` and consumes the signer nonce before changing direct-mode state. A compromised `BatchExecutor` caller can submit calldata to a wallet, but the wallet still rejects these direct-mode calls unless the calldata includes a valid registered-owner signature.

Consequently, the documented compromised-caller scenario is too broad. A leaked `authorizedCaller` cannot systematically start direct mode, cancel a pending direct-mode entry, or force a direct-mode exit across wallets by itself. Those actions require wallet-owner authorization. The real caller-only analysis should focus on selectors that truly do not require owner authorization, such as `executeRecovery()` after the recovery delay and on selectors with different non-owner controls, such as executor-signed allowance spends.

The same selector inventory also contains stale ABI strings. For example, `executorApproveAndExternalCall` is missing the `target` and `value` arguments, both external approval signatures omit `approvalDeadline` and `revokePreAuthorizedHash` omits the `hash` argument. This makes the table unreliable for threat modeling or automated selector checks.

## — RECOMMENDATION

Update `THREAT_MODEL.md` so it describes the current code instead of treating every selector reachable through `batchExecute` as successful without owner authorization. In the

**I-10** Threat model overstates direct-mode caller risk

CONTINUED

selector table, keep `initiateDirectMode`, `cancelDirectMode`, `initiateExitDirectMode` and `cancelExitDirectMode` marked as `Owner sig only`. In the compromised-`authorizedCaller` section, remove those direct-mode functions from the "no owner signature requirement" list and add a note that `batchExecute` can route the calldata, but the wallet will revert unless the calldata includes a valid registered-owner signature.

Keep `executeRecovery()` in the no-owner-signature list only with its recovery-delay precondition. For `spendDailyAllowance` and `spendDailyAllowanceEth`, state that a compromised caller also needs valid executor-signed spend calldata and must pass the `approvedRecipients` policy. Finally, regenerate or manually correct the stale ABI strings for `instantSetAllowance`, `executorApproveAndExternalCall`, `externalApprove`, `externalApproveAndPreAuthorizeHash` and `revokePreAuthorizedHash` so the selector inventory matches the compiled contracts.

**— RESOLUTION**

Status: Acknowledged.

INFORMATIONAL **I-11**

ACKNOWLEDGED

# Overloaded DailyLimitExceeded revert error

CATEGORY Best Practices  
LOCATION src/libraries/LibAllowance.sol  
REVIEW Main Review

## DESCRIPTION

`spendDailyAllowance` reuses a single error, `DailyLimitExceeded`, for two semantically distinct failure conditions:

```
function spendDailyAllowance(address token, uint256 amount) internal {  
    AllowanceStorage storage als = allowanceStorage();  
    uint256 limit = als.dailyLimits[token];  
    if (limit == 0) revert DailyLimitExceeded(); // token never configured / blocked  
    ...  
    if (spending.amountSpent + amount > limit) revert DailyLimitExceeded(); // cap actually  
    exceeded  
}
```

The first branch fires when the token has no daily limit set, where `0` is the default-blocked state (the executor cannot spend until the owner sets a limit via `instantSetAllowance`). The second branch fires when a configured limit is genuinely exceeded. Both revert with the same selector, so the two conditions are indistinguishable to any caller.

## RECOMMENDATION

Introduce a distinct error for the unconfigured/blocked case, for example `DailyLimitNotSet(address token)` (or `TokenSpendingBlocked`), and revert with it in the `limit == 0` branch, reserving `DailyLimitExceeded` for the case where a configured limit is actually surpassed. This lets off-chain systems and integrators react correctly to each condition and aligns the spend path with the state distinction the view functions already expose.

## RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-12

ACKNOWLEDGED

# Passkey signer docs use qx instead of key hash

CATEGORY Documentation  
LOCATION docs/EIP712\_REFERENCE.md  
REVIEW Main Review

## DESCRIPTION

The EIP-712 reference tells integrators to encode a passkey signer as `qx`, the P256 public key X coordinate. The wallet does not use `qx` as the passkey signer key. `LibAuth._passkeyKey` derives the key as `keccak256(abi.encode(qx, qy))` and `AuthFacet` rejects every signed operation when the verified signer key does not equal the `signer` field in calldata.

The same incorrect guidance appears in the user and operator guides. An integration that follows those docs will query `nonce(bytes32)` with the wrong key, build typed data with `signer = qx` and produce signatures that revert with `SignerMismatch`. This affects passkey-signed signer management and every other owner-signed wallet operation that uses the shared `signer` field.

Consequently, wallets, SDKs or emergency runbooks built from the documentation cannot execute passkey-authorized actions when they are needed.

## RECOMMENDATION

Update every signer-key reference to state that passkey signer keys are `keccak256(abi.encode(qx, qy))`. Keep the owner encoding as `bytes32(uint256(uint160(ownerAddress)))`.

Consider exposing a small view helper that returns the signer key for a stored passkey or for supplied `(qx, qy)` coordinates, so off-chain tooling does not need to duplicate the encoding rule.

## RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-13

ACKNOWLEDGED

# Docs claim delay is signed in config

CATEGORY Documentation  
LOCATION docs/EIP712\_REFERENCE.md  
REVIEW Main Review

## DESCRIPTION

The EIP-712 reference says the `SetRecoveryConfig` `configHash` is the hash of the "admins list + delay". The current wallet does not sign the recovery delay in this message. `RecoveryFacet.setRecoveryConfig` computes `configHash` as `keccak256(abi.encode(admins))`, then signs that hash together with `signer`, `nonce` and `deadline` through `LibRecovery.hashSetRecoveryConfig`.

The recovery delay is not configurable per wallet. It is the fixed `LibRecovery.RECOVERY_DELAY` constant, returned by `getRecoveryConfig()` and enforced when `executeRecovery()` checks the pending recovery timestamp. Therefore, this is not a runtime authorization bypass in the current code.

Essentially, the signing reference describes a value that the wallet never verifies. A client built from the docs may display a recovery-delay value as part of the signed recovery config, even though the signature only commits to the admin list. A client that actually signs `keccak256(abi.encode(admins, delay))` will produce a signature the wallet rejects.

## RECOMMENDATION

Update the EIP-712 reference to state that `configHash = keccak256(abi.encode(admins))`. Add an explicit note that the 10-day recovery delay is the fixed `LibRecovery.RECOVERY_DELAY` contract constant and is not part of the `SetRecoveryConfig` signed payload. If a future upgrade makes the delay configurable, update both the typed-data spec and the on-chain hashing code to include the exact delay value.

## RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-14

ACKNOWLEDGED

# Parent preauth can authorize child ops

|          |                             |
|----------|-----------------------------|
| CATEGORY | Unexpected Behavior         |
| LOCATION | src/facets/ERC1271Facet.sol |
| REVIEW   | Main Review                 |

## DESCRIPTION

`ERC1271Facet.isValidSignature` returns the ERC-1271 magic value for any hash stored in `LibPreAuthorizedHash`, even when the supplied signature is empty.

`externalApproveAndPreAuthorizeHash` can store any nonzero `orderHash` and that hash is treated as an opaque external order hash. The stored value is not tagged with a purpose, an expected verifier or the wallet operation it is allowed to authorize.

This becomes unsafe when one Ethenapay wallet is registered as an EIP-1271 owner of another wallet. The child wallet's `LibAuth._verifyEIP1271` checks that the parent wallet address is one of its owners, then asks the parent wallet whether the child operation digest is a valid signature. If the parent wallet has preauthorized that same digest as an `orderHash`, the parent returns the ERC-1271 magic value. The child wallet then treats the parent response as owner approval for its own operation.

For example:

- A user or business uses a parent wallet as the smart-contract owner of a child wallet that holds operational funds.
- An integration asks the parent wallet to preauthorize an opaque `orderHash` through `externalApproveAndPreAuthorizeHash`.
- The opaque hash is chosen to equal the EIP-712 digest of a child-wallet operation, such as `executorEthWithdrawal` to a recipient controlled by the integrator or attacker.
- The parent signer and executor authorize the preauthorization transaction, believing it is an external order authorization for the parent wallet.
- Later, the child wallet receives an EIP-1271 owner signature that contains only the parent wallet address and no live contract signature.
- The child calls `isValidSignature` on the parent wallet. Because the digest was preauthorized, the parent returns the magic value and the child accepts it as owner approval for the withdrawal.

This requires the child wallet to use the parent wallet as an EIP-1271 owner. It also requires the parent signer and executor to authorize the chosen hash, plus the child executor signature for the child operation. The issue is that the parent preauthorization is not

**I-14** Parent preauth can authorize child ops

CONTINUED

limited to the external protocol context shown to the parent signer. The same hash can be reused as authorization for a different wallet's internal operation.

**— RECOMMENDATION**

Separate external-order preauthorizations from EIP-1271 owner signatures used by wallet-to-wallet authentication. Store enough context with each preauthorized hash to restrict who may consume it, such as an authorization purpose and expected verifier. Then make `isValidSignature` return the magic value for a preauthorized hash only when the caller matches that context.

If wallet proxies are allowed to be owners of other wallets, also require a non-empty live contract signature when `LibAuth._verifyEIP1271` verifies an Ethenapay wallet owner. A preauthorized hash on the owner wallet should not by itself satisfy authorization for a different wallet's internal operation.

**— RESOLUTION**

Status: Acknowledged.

## INFORMATIONAL I-15

ACKNOWLEDGED

# Recovery bypasses signer count caps

CATEGORY DoS  
LOCATION src/libraries/LibAuth.sol  
REVIEW Main Review

## DESCRIPTION

Normal signer additions enforce `MAX_PASSKEYS` and `MAX_OWNERS` so the wallet's linear auth scans stay bounded. The recovery direct-add helpers skip those caps. `addPasskeyDirect` pushes a passkey regardless of the current passkey count and `addOwnerDirect` pushes an owner regardless of the current owner count.

As a result, a recovery execution can take a wallet above the advertised 10-passkey or 10-owner limits. Repeated recoveries can keep growing the signer arrays over time. This makes the gas cost of `isOwner`, `hasPasskey`, signer removal and WebAuthn verification scale beyond the bounds assumed elsewhere in `LibAuth`.

The issue requires a recovery admin and the 10-day recovery delay, so it is not an immediate public denial of service. It is still a code-level exception to the signer-count invariant. A wallet that accumulates many recovery-added signers can become more expensive or unreliable to operate, especially for WebAuthn signatures that may test each stored passkey before finding the matching key.

## RECOMMENDATION

Keep recovery within a bounded signer model. If a wallet is already at the signer cap, recovery should replace an existing signer, require an explicit signer-removal step, or use a separate bounded emergency slot that is promoted only after a stale signer is removed.

If bypassing the caps remains intentional, update the invariants and user-facing limits to state that recovery can exceed them. Add an operational cap or monitoring alert so repeated recovery executions cannot silently grow signer arrays until normal auth operations become too expensive.

## RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-16

RESOLVED

# Upgrade omits approval view selectors

|          |                                            |
|----------|--------------------------------------------|
| CATEGORY | Configuration                              |
| LOCATION | script/examples/AddPreAuthorizeFacet.s.sol |
| REVIEW   | Main Review                                |

## DESCRIPTION

`AddPreAuthorizeFacet` schedules the new preauthorization facet with only `externalApproveAndPreAuthorizeHash`, `revokePreAuthorizedHash` and `clearStaleApproval`. It also repoints `isValidSignature`, but it does not register `currentApprovalOwner` or `approvalEntry`.

Fresh deployments include those two view selectors in `DeployBeaconFactory`, so this only affects factories upgraded with the example post-deploy script. After such an upgrade, the wallet can still create and clear deadline-gated approvals, but off-chain monitors, keepers and operators cannot query the current allowance owner or the stored approval entry through the wallet proxy. Calls to those selectors revert with `FunctionNotFound`.

This does not directly move funds. The risk is weaker state-approval observability after an upgrade. Cleanup automation that depends on the view functions may fail to discover or verify expired approvals.

## RECOMMENDATION

Register `ExternalApproveAndPreAuthorizeHashFacet.currentApprovalOwner.selector` and `ExternalApproveAndPreAuthorizeHashFacet.approvalEntry.selector` in the post-deploy upgrade script. Update the selector array length and the `AddPreAuthorizeFacet` tests so the legacy-upgrade scenario asserts that both view selectors route after the cut.

## RESOLUTION

Status: Resolved.

INFORMATIONAL **I-17**

RESOLVED

**executeRecovery breaks reentrancy invariant**

CATEGORY Informational  
LOCATION src/facets/RecoveryFacet.sol  
REVIEW Main Review

**DESCRIPTION**

Every other externally-callable, state-mutating facet function brackets its body with `LibReentrancyGuard.enforceNonReentrant()` / `clearReentrantFlag()` (withdrawals, allowance set/spend, external calls, approvals, all Auth initiate/approve/cancel, all Recovery owner-ops, all DirectMode ops). The only exceptions are `AllowanceFacet.invalidateSpendNonces` (intentionally permissionless and purely protective) and `executeRecovery`, which mutates the signer set (`addOwnerDirect` / `addPasskeyDirect`) but carries no guard.

**RECOMMENDATION**

Add `enforceNonReentrant()` at entry and `clearReentrantFlag()` on every exit path (including the early `return`).

**RESOLUTION**

Status: Resolved.

## INFORMATIONAL I-18

RESOLVED

# Executor wallet preauth satisfies executor sig

|          |                           |
|----------|---------------------------|
| CATEGORY | Warning                   |
| LOCATION | src/libraries/LibAuth.sol |
| REVIEW   | Main Review               |

## DESCRIPTION

`verifyExecutorSignature` accepts the current factory executor through `OpenZeppelin SignatureChecker`. If the factory executor is an Ethenapay wallet proxy, the executor check calls that wallet's `ERC1271Facet.isValidSignature`.

The preauthorized-hash behavior in `ERC1271Facet.isValidSignature` is intentional. The repository documents preauthorized hashes as values the wallet will accept when `isValidSignature` is called. `LibPreAuthorizedHash` also says the feature exists so the wallet can return ERC-1271 magic for a specific hash without requiring a fresh signature at verification time. The intended example is a 1inch Fusion+ style order: Alice signs one wallet authorization, the wallet approves the router, the wallet stores the external `orderHash` and a resolver later calls `wallet.isValidSignature(orderHash, "")`.

The warning is that the same ERC-1271 answer can be consumed by a different authorization role. For example, Alice owns the target wallet. Mallory is only the recipient and transaction submitter. The executor is the protocol co-signer that should provide the second approval for `executorEthWithdrawal`. A separate Ethenapay wallet is later rotated into the global executor role. Before that rotation, this executor wallet stores Alice's withdrawal digest as a preauthorized `orderHash`.

After the rotation, Alice's wallet checks whether the current executor approved the withdrawal. Since the executor is now a wallet proxy, `SignatureChecker` calls `executorWallet.isValidSignature(withdrawalDigest, "")`. The executor wallet returns ERC-1271 magic because that digest is already stored in `LibPreAuthorizedHash`. Alice's wallet then accepts the empty `executorSig` and executes the withdrawal because Alice's owner signature is valid.

This does not show that Mallory can steal funds without Alice. Alice still signs the exact withdrawal parameters, including recipient, amount, nonce and deadline. Mallory cannot change those values. The concern is limited to the executor-signature check: a generic preauthorized hash on a contract executor can satisfy that check for another wallet operation.

**I-18** Executor wallet preauth satisfies executor sig

CONTINUED

**RECOMMENDATION**

Document the intended scope of preauthorized hashes. If a preauthorized hash is meant to be a valid ERC-1271 signature for every caller and purpose, document that executor-wallet configurations inherit that behavior.

If executor approval is meant to remain a separate authorization layer, do not let generic wallet ERC-1271 preauthorizations satisfy executor signatures. Require a dedicated executor-verification interface or an ERC-1271 signature mode that cannot return success from `LibPreAuthorizedHash`.

At minimum, reject Ethenapay wallet proxies as factory executors unless they implement a separate executor-only validation facet. A stronger fix is to bind each preauthorized hash to an explicit purpose and verifier, then make `ERC1271Facet.isValidSignature` return the magic value only when `msg.sender` matches the stored verifier and the stored purpose is valid for that caller.

**RESOLUTION**

Status: Resolved.

## INFORMATIONAL I-19

PARTIALLY RESOLVED

# BatchExecutor deploy ignores initial owner

|          |                                  |
|----------|----------------------------------|
| CATEGORY | Documentation                    |
| LOCATION | script/DeployBatchExecutor.s.sol |
| REVIEW   | Main Review                      |

## DESCRIPTION

`DeployBatchExecutor.s.sol` always deploys the executor with `deployer` as the constructor `initialOwner`. The script never reads the `INITIAL_OWNER` value shown in the replacement guide.

Consequently, an operator following the guide can believe the new `BatchExecutor` is owned by the multisig while the deployer EOA actually controls `addCaller`, `removeCaller`, `addRecipient`, `removeRecipient`, `addBlocked`, `removeBlocked`, `pause` and `unpause`. The same script also auto-registers the deployer as an authorized caller. If the handoff is missed during a replacement, a hot deployer key keeps both administrative control and batch-submission authority over the new executor.

The main deployment guide later includes separate ownership-transfer steps and the validation scripts can catch the wrong owner if they are run with the correct expected owner. The replacement guide does not include that handoff, so the script behavior is still a concrete operational security risk for BatchExecutor replacements.

## RECOMMENDATION

Read an explicit owner from the environment and pass that value into the constructor or remove `INITIAL_OWNER` from the guide and make the two-step BatchExecutor ownership transfer a required step in every replacement runbook.

If the owner is not the deployer, split deployment from initial configuration. The owner should perform caller, recipient and blacklist setup or the script should clearly stop after deployment and instruct the owner multisig to execute the configuration transactions.

## RESOLUTION

Status: Partially resolved.

## INFORMATIONAL I-20

RESOLVED

# Recovery reset event misreports admin set

CATEGORY Best Practices

LOCATION src/facets/RecoveryFacet.sol

REVIEW Main Review

## DESCRIPTION

`setRecoveryConfig` treats an empty `admins` array as "reset to factory defaults." In that branch it deletes the custom admins, sets `hasCustomConfig = false`, and emits:

```
// admins.length == 0 branch
delete rs.customRecoveryAdmins;
rs.hasCustomConfig = false;
emit RecoveryConfigChanged(address(this), 0, configHash); // configHash =
keccak256(abi.encode(admins)) = hash of []
```

The event reports `adminCount = 0` and `configHash = keccak256(abi.encode([]))`. But after this call the wallet's effective recovery admin set is the factory defaults, not an empty set.

`LibRecovery.getEffectiveAdmins` returns `factory.getDefaultRecoveryAdmins()` whenever `hasCustomConfig == false` and the wallet is not opted out:

```
if (rs.optedOut) return new address[](0);
if (rs.hasCustomConfig) return rs.customRecoveryAdmins;
return IUserProxyFactory(factory).getDefaultRecoveryAdmins(); // <-- effective set after a
reset
```

So the on-chain getter (`getRecoveryConfig`) correctly returns the factory defaults, but the emitted event tells a different story.

## RECOMMENDATION

Make the event reflect the effective configuration after a reset. Either emit the factory default admin count and a hash of the effective set in the reset branch, or add a distinct event (e.g.

```
RecoveryConfigResetToDefaults(wallet) ).
```

## RESOLUTION

Status: Resolved.

MEDIUM **M-01**

ACKNOWLEDGED

# Authorized deployer seizes funded UUID wallets

|          |                                |
|----------|--------------------------------|
| CATEGORY | Unexpected Behavior            |
| LOCATION | src/UserProxyFactoryBeacon.sol |
| REVIEW   | Remediation Review             |

## DESCRIPTION

Wallet addresses are derived only from `walletUuid` and the beacon proxy init code. The initial signer is not part of the CREATE2 salt. Instead, an authorized deployer supplies the first passkey or owner when `deployWalletWithPasskey`, `deployWalletWithOwner` or `deployWalletWithBoth` is called.

Here, “authorized deployer” does not mean any arbitrary address. It means the factory `owner()` or an address with `isDeployer[account] == true`, because the deployment functions are guarded by `onlyDeployer`. The factory also grants deployer status to the initial `executor` during initialization, so that executor key is custody-critical for undeployed wallets that may already hold funds.

Consequently, an authorized deployer that knows a funded `walletUuid` can deploy that wallet with attacker-controlled signer data. Any assets already sent to the predicted address then become controlled by the deployer-selected signer. This breaks the non-custodial assumption for undeployed counterfactual wallets that users or integrators can pre-fund before initialization.

This is especially sensitive because the factory grants deployer status to the initial `executor` during initialization.

For example:

1. A user or integrator computes the wallet address from `walletUuid`.
2. They send ETH or tokens to that predicted address before deployment.
3. An authorized deployer learns that `walletUuid`.
4. Instead of deploying with the real user’s signer, the deployer calls `deployWalletWithOwner(walletUuid, attackerOwner)`.
5. The wallet is deployed at the already-funded address.
6. `WalletDiamond.initializeWithOwner()` registers `attackerOwner` as the wallet owner:

**M-01** Authorized deployer seizes funded UUID wallets

CONTINUED

```
function initializeWithOwner(address owner) external {
    if (_isInitialized()) revert AlreadyInitialized();
    _setInitialized();
    LibFactory.setFactory(msg.sender);
    LibAuth.initializeWithOwner(owner);
    _applyDefaultAllowances();
    emit WalletInitialized(msg.sender);
    emit OwnerAdded(owner);
}
```

1. The real user cannot redeploy the wallet with their own signer because the UUID is now used and `walletProxies[walletUuid]` is set.
2. The attacker-controlled signer can enter direct mode and withdraw the funds. Direct mode is signer-authorized by `initiateDirectMode`:

```
function initiateDirectMode(bytes32 signer, uint256 nonce, uint256 deadline, bytes calldata
ownerSig) external {
    LibReentrancyGuard.enforceNonReentrant();
    if (block.timestamp > deadline) revert DirectModeExpired();
    bytes32 structHash = LibDirectMode.hashInitiateDirectMode(signer, nonce, deadline);
    bytes32 digest = LibEIP712.toTypedDataHash(structHash);
    bytes32 signerKey = LibAuth.verifyOwnerSignature(digest, ownerSig);
    if (signerKey != signer) revert LibAuth.SignerMismatch(signer, signerKey);
    LibAuth._useCheckedNonce(signerKey, nonce);
    LibAuth.enforceNotPendingRemoval(signerKey);
    uint64 activatesAt = LibDirectMode.initiateDirectMode();
    emit DirectModeInitiated(activatesAt);
    LibReentrancyGuard.clearReentrantFlag();
}
```

The relevant `directEthWithdrawal` logic then verifies the same signer model and sends ETH:

```
function directEthWithdrawal(
    bytes32 signer,
    address to,
    uint256 amount,
    uint256 nonce,
    uint256 deadline,
    bytes calldata ownerSig
) external {
```

**M-01** Authorized deployer seizes funded UUID wallets

CONTINUED

```
LibReentrancyGuard.enforceNonReentrant();
if (!LibDirectMode.isDirectMode()) {
  revert LibDirectMode.DirectAccessNotAvailable();
}
if (block.timestamp > deadline) revert WithdrawalExpired();
if (to == address(0)) revert ZeroAddress();
if (amount == 0) revert ZeroAmount();
bytes32 structHash = _hashDirectWithdrawal(
  signer,
  address(0),
  to,
  amount,
  nonce,
  deadline
);
bytes32 digest = LibEIP712.toTypedDataHash(structHash);
bytes32 signerKey = LibAuth.verifyOwnerSignature(digest, ownerSig);
if (signerKey != signer) revert LibAuth.SignerMismatch(signer, signerKey);
LibAuth._useCheckedNonce(signerKey, nonce);
LibAuth.enforceNotPendingRemoval(signerKey);
(bool success,) = to.call{value: amount}("");
if (!success) revert EthTransferFailed();
}
```

**— RECOMMENDATION**

Bind deployment to the intended first signer set. The safest fix is to include a commitment to the initial signer data in the CREATE2 salt or require a user-signed deployment authorization that names the `walletUuid`, factory, chain ID and initial signer set.

If UUID-only addresses must remain unchanged, treat the factory owner and every `isDeployer` address, including the initial executor, as custody-critical for undeployed funded wallets. Keep deployer authority away from hot executor keys and add operational checks that prevent a deployer from choosing arbitrary initial signers for a funded UUID.

**— RESOLUTION**

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

# Prefunded wallets inherit mutable defaults

CATEGORY Unexpected Behavior  
LOCATION src/WalletDiamond.sol  
REVIEW Remediation Review

## DESCRIPTION

Wallet addresses are predictable before deployment, but the daily allowances installed at deployment are not bound to that prediction. `initializeWithPasskey`, `initializeWithOwner` and `initializeWithBoth` all call `_applyDefaultAllowances`. That helper reads the factory's current default daily limits and writes them into the wallet during initialization.

The factory owner can change default daily limits before a wallet is deployed. If a user or integrator computes a wallet address from `walletUuid` and sends ETH or tokens to that address before deployment, the wallet can later be deployed with a different default allowance configuration than the one expected when the funds were sent. This does not require the attacker-controlled signer scenario from the UUID deployment issue. The wallet can be initialized with the correct user signer and still inherit mutable executor spending limits chosen after the address was prefunded.

For example:

Day 1:  
The user predicts a wallet address from `walletUuid`.  
Factory defaults are harmless, such as USDe daily limit = 0.  
The user sends funds to the predicted address.

Day 2:  
The factory owner changes default daily limits.  
Future wallets now receive USDe daily limit = 100,000.

Day 3:  
The wallet is deployed at the predicted address with the correct user signer.  
It inherits the Day 2 limits, not the limits expected when funds were sent.

Consequently, prefunded counterfactual wallets can start life with executor daily allowances the user did not approve for that wallet. The executor can then spend up to those inherited limits before the user notices and overrides them.

**M-02** Prefunded wallets inherit mutable defaults

CONTINUED

**— RECOMMENDATION**

Do not apply mutable factory defaults to prefunded counterfactual wallets without a deployment-time commitment. The safest option is to initialize new wallets with zero daily limits and require the owner to set limits after deployment.

If defaults must remain, bind the expected default-limit hash into wallet deployment. For example, make deployment take an `expectedDefaultLimitsHash` and revert if the factory defaults changed between address prediction and deployment. Documentation and integrations should also warn users not to prefund predicted wallet addresses unless the wallet is already deployed or the default allowance configuration is pinned.

**— RESOLUTION**

Status: Acknowledged.

MEDIUM **M-03**

ACKNOWLEDGED

# Preauthorized hashes ignore pending removal

CATEGORY Validation  
LOCATION `src/libraries/LibPreAuthorizedHash.sol`  
REVIEW Remediation Review

## DESCRIPTION

A preauthorized hash is a stored approval for a specific external order. When an external protocol later calls `isValidSignature(hash, sig)`, the wallet returns the ERC-1271 magic value if

`LibPreAuthorizedHash.isAuthorized(hash)` says that stored approval is still valid.

`LibPreAuthorizedHash.isAuthorized` checks whether the entry expired, whether the authorizing signer is still registered and whether the executor epoch is still current. It does not check whether the authorizing signer is pending removal.

This creates a gap in the pending-removal quarantine. Normal owner-authenticated functions call `LibAuth.enforceNotPendingRemoval`, so a signer queued for removal cannot keep authorizing sensitive wallet actions. New preauthorizations are also blocked because `externalApproveAndPreAuthorizeHash` checks `enforceNotPendingRemoval` before storing a hash. However, hashes stored before the signer was queued for removal remain valid.

For example, signer A can preauthorize an external order hash while it is healthy. If the user later suspects signer A is compromised and queues it for removal, signer A is still registered until the removal cooldown ends. During that cooldown, an external protocol can ask the wallet whether the old order hash is valid. The wallet still returns the ERC-1271 magic value because `isAuthorized` only checks that signer A is registered, not that signer A is pending removal.

Consequently, queuing a signer for removal does not suspend external orders that signer already authorized. This weakens pending removal as an emergency containment step for a suspected compromised signer.

## RECOMMENDATION

Make `LibPreAuthorizedHash.isAuthorized` return false when `entry.signerKey` is pending removal:

**M-03** Preauthorized hashes ignore pending removal

CONTINUED

```
if (LibAuth.isPendingRemoval(entry.signerKey)) {  
    return false;  
}
```

The check should match the pending owner and passkey removal logic already used by owner-authenticated state changes. For stronger containment, revoke signer-scoped preauthorizations and clear their paired deadline-gated approval entries when a signer removal is initiated or executed.

**— RESOLUTION**

Status: Acknowledged.

MEDIUM **M-04**

ACKNOWLEDGED

# Approve and call can leave approvals

|          |                                          |
|----------|------------------------------------------|
| CATEGORY | Trust Assumptions                        |
| LOCATION | src/facets/ExecutorExternalCallFacet.sol |
| REVIEW   | Remediation Review                       |

## DESCRIPTION

`executorExternalCall` blocks several approval-mutating selectors before it calls an external target. `executorApproveAndExternalCall` does not apply the same guard to its `data`.

That function grants a temporary allowance to the supplied `spender`, calls arbitrary `target.call(data)`, then clears only the allowance for the original `(token, spender)` pair. If `data` calls a token or operator contract and creates a different approval, that second approval is not cleared.

For example, the signed payload can set `token` and `spender` to a legitimate router while setting `target` to an ERC-20 token and `data` to `approve(attacker, type(uint256).max)`. The function will clear the router allowance after the call, but the attacker allowance remains live. The same issue applies to approval-like functions that authorize a different spender or operator.

This requires both owner and executor signatures. The impact is still meaningful because the function's cleanup gives users and integrators the impression that the approval granted for the external call cannot persist after execution.

## RECOMMENDATION

Apply the same selector guard used by `executorExternalCall` before `executorApproveAndExternalCall` performs `target.call(data)`.

A safer fix is to replace the short blocklist with an allowlist of supported targets and selectors. If generic calls remain supported, block standard ERC-20 approvals, legacy ERC-20 approval variants, ERC-721 and ERC-1155 operator approvals and known protocol-specific approval functions.

## RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

# Immediate RP ID hash update can brick passkeys

|          |                                |
|----------|--------------------------------|
| CATEGORY | Compatibility                  |
| LOCATION | src/UserProxyFactoryBeacon.sol |
| REVIEW   | Remediation Review             |

## DESCRIPTION

`setExpectedRpIdHash` lets the factory owner replace the global WebAuthn RP ID hash immediately. There is no schedule, delay, cancellation window or grace period that accepts both the old and new RP ID hashes.

`LibAuth` reads this factory value during WebAuthn verification and rejects authenticator data whose first 32 bytes do not match the configured hash. Therefore, a typo or malicious owner action can instantly make existing passkey assertions invalid for every wallet using the previous RP ID. Passkey-only wallets would lose the ability to authorize direct mode, direct withdrawals, signer management and recovery cancellation until the value is corrected or another signer type is available.

Existing tests confirm that a configured mismatched RP ID hash causes WebAuthn verification to revert with `InvalidRpIdHash`.

## RECOMMENDATION

Put RP ID hash changes behind the same governance delay used for other global wallet-risk changes. Use a two-step process with `scheduleExpectedRpIdHash`, `executeExpectedRpIdHash` and `cancelExpectedRpIdHash`. For domain migrations, support a grace period where both the old and new RP ID hashes are accepted.

## RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

# Old executor keeps deployer role after rotation

|          |                                |
|----------|--------------------------------|
| CATEGORY | Trust Assumptions              |
| LOCATION | src/UserProxyFactoryBeacon.sol |
| REVIEW   | Remediation Review             |

## DESCRIPTION

The factory has two separate privileges. `executor` is the key or contract used to co-sign executor-authorized wallet operations. `isDeployer[address]` is a separate allowlist for addresses that can deploy new wallets.

During initialization, the factory automatically grants deployer status to the initial executor by setting `isDeployer[_executor] = true`. Later, `executeExecutorChange` rotates `executor` and increments `executorEpoch`, but it does not update `isDeployer`.

Consequently, an old executor loses the ability to co-sign new executor operations, but it can still deploy wallets if it was previously left in `isDeployer`. The deployment functions check `onlyDeployer`, not whether `msg.sender` is the current executor. They also let the deployer choose the first owner or passkey for the new wallet.

This matters because wallet addresses are deterministic from `walletUuid`; the initial signer is not part of the address. If a user or integrator sends assets to a predicted wallet address before deployment, a stale old executor that knows the `walletUuid` can deploy that wallet with an attacker-controlled signer. The assets at the predicted address then become controlled by the attacker-selected signer.

## RECOMMENDATION

Do not couple deployment authority to the executor implicitly. Remove the automatic `isDeployer[_executor] = true` grant if executor keys are not meant to deploy wallets.

If the executor must also be a deployer, update deployer roles atomically during executor rotation:

```
isDeployer[oldExecutor] = false;  
isDeployer[newExecutor] = true;
```

**L-02** Old executor keeps deployer role after rotation

CONTINUED

If deployers are meant to be managed independently, document that separation. The executor rotation runbook should then explicitly revoke the old executor's deployer role in the same governance operation.

**— RESOLUTION**

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

# Emergency revokes require executor epoch

|          |                                          |
|----------|------------------------------------------|
| CATEGORY | Unexpected Behavior                      |
| LOCATION | src/facets/ExecutorExternalCallFacet.sol |
| REVIEW   | Remediation Review                       |

## DESCRIPTION

`executorEpoch` is the factory's version number for the executor key. It is useful for executor-signed actions because old executor signatures should stop working after executor rotation.

`emergencyPermit2Lockdown` and `emergencyRevokeApproval` are different. They are documented as owner-only emergency actions and they do not verify an executor signature. Even so, both functions require the owner to sign the current `executorEpoch`, then both call `_enforceExecutorEpoch`.

Consequently, an unrelated executor rotation can invalidate a still-fresh owner signature for emergency cleanup. For example, a user may sign an emergency Permit2 lockdown while `executorEpoch` is `0`. If governance rotates the executor before that transaction is submitted, the factory increments `executorEpoch` to `1`. The lockdown then reverts with `ExecutorEpochMismatch(0, 1)`, even though the executor was never meant to authorize the lockdown.

This is a bad failure mode for revocation functions. These functions exist so an owner can quickly cut off external approvals without executor cooperation during direct mode or another incident. Tying them to `executorEpoch` can delay that protective cleanup and leave the approval live until the user signs and submits a new payload.

## RECOMMENDATION

Remove `executorEpoch` from owner-only emergency revocation payloads. These functions should not call `_enforceExecutorEpoch` because the executor does not sign or approve them. Keep the owner signer, nonce and deadline checks.

If the protocol wants emergency signatures to expire on a governance boundary, use a dedicated revocation epoch for owner-only emergency actions. Do not reuse the executor epoch unless the executor also signs the payload.

---

**L-03** Emergency revokes require executor epoch

CONTINUED

---

— **RESOLUTION**

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

## Batch executor can disable blocklist

|          |                                |
|----------|--------------------------------|
| CATEGORY | Unexpected Behavior            |
| LOCATION | src/UserProxyFactoryBeacon.sol |
| REVIEW   | Remediation Review             |

### DESCRIPTION

Direct daily allowance spends rely on `factory.batchExecutor()` for the recipient blocklist.

`AllowanceFacet.spendDailyAllowance` and `spendDailyAllowanceEth` do not store blocklist state in the wallet. They read the current `batchExecutor` address from the factory, then ask that contract whether the recipient is blocked.

If `batchExecutor()` returns `address(0)`, the wallet skips the blocklist check completely. If it returns a contract, the wallet trusts that contract's `blocked(address)` response.

`setBatchExecutor` can change this address immediately. It is `onlyOwner`, but it has no timelock. It also explicitly allows `address(0)`, which disables the wallet-level blocklist. For nonzero addresses, the only validation is that the target has code and exposes a callable `blocked(address)` function. The factory does not require the target to be the real `BatchExecutor` or a governed recipient-policy contract.

For example:

```
Normal state:
factory.batchExecutor = real BatchExecutor
The wallet asks the real BatchExecutor whether the recipient is blocked.
Blocked recipients are rejected.
After setBatchExecutor(address(0)):
factory.batchExecutor = address(0)
The wallet sees no blocklist contract and skips the check.
Blocked recipients can receive direct allowance spends.
```

The same issue exists with a fake replacement contract. A compromised or mistaken factory owner can point `batchExecutor` at a contract whose `blocked(address)` function always returns `false`. Existing wallets then keep calling a blocklist hook, but the hook no longer enforces the intended policy.

This is not permissionless. It requires factory-owner authority or a compromised owner. The issue is that the change is instant, global and affects existing wallets' direct allowance-spend protection.

---

**L-04** Batch executor can disable blocklistCONTINUED

---

**— RECOMMENDATION**

Put `setBatchExecutor` behind the same scheduled governance process used for executor and facet changes. After production setup, do not allow `address(0)` unless the change is timelocked and clearly monitored.

Consider separating the recipient-policy registry from the batch relay contract. The wallet should read blocklist state from a dedicated, governed policy contract rather than from a mutable relay address.

**— RESOLUTION**

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

# Pending recovery initiator can become executor

|          |                              |
|----------|------------------------------|
| CATEGORY | Validation                   |
| LOCATION | src/facets/RecoveryFacet.sol |
| REVIEW   | Remediation Review           |

## DESCRIPTION

`RecoveryFacet.initiateRecovery` enforces role separation when recovery starts. The caller must be a recovery admin and the call reverts if that same address is the current factory executor. This prevents the executor from using recovery-admin authority to add a wallet signer.

Recovery is delayed for 10 days. During that delay, the factory executor can be rotated. `executeRecovery` checks that `rs.pendingInitiator` is still a recovery admin and that the recovery admin config hash has not changed, but it does not check whether that same pending initiator has become the current executor.

For example, admin `A` can initiate recovery while the executor is `E`. If the factory executor is later rotated to `A` before the timelock ends, `executeRecovery` can still complete the pending recovery because `A` remains in the admin list and the admin list itself is unchanged. The result contradicts the rule enforced by `initiateRecovery`: the current executor should not be able to act through recovery-admin authority.

The existing `owner == currentExecutor` invalidation only covers an address signer that would be added by the recovery. It does not cover the recovery initiator and it does not cover passkey recoveries. Consequently, the executor and recovery-admin roles can overlap for an already pending recovery after a factory-level role rotation.

## RECOMMENDATION

Recheck the pending recovery initiator against the current executor inside `executeRecovery`. If `rs.pendingInitiator == IUserProxyFactory(factory).executor()`, clear the pending recovery and emit `RecoveryInvalidated`.

Apply the same role-separation check to any other recovery-admin role that should not remain valid after factory-level role rotation.

---

**L-05** Pending recovery initiator can become executor

CONTINUED

---

— **RESOLUTION**

Status: Acknowledged.

LOW **L-06**

ACKNOWLEDGED

# Default recovery admins can become unusable

|          |                                |
|----------|--------------------------------|
| CATEGORY | Validation                     |
| LOCATION | src/UserProxyFactoryBeacon.sol |
| REVIEW   | Remediation Review             |

## DESCRIPTION

`scheduleDefaultRecoveryAdmins` validates the pending default recovery admins against the executor that is current at scheduling time. `executeDefaultRecoveryAdmins` later copies the pending admins into `_defaultRecoveryAdmins` without re-running that validation.

Executor changes are scheduled and executed separately. Therefore, the factory can schedule a recovery-admin set that is valid against the old executor, rotate the executor before the recovery-admin update is executed and then install a default admin that equals the current executor. The same stale configuration can also arise when the executor is rotated after defaults are already active.

The wallet recovery code now rejects the current executor as a recovery admin when `initiateRecovery` is called. That prevents role collapse, but it also means the configured default admin may be unusable. Wallets that rely only on factory default recovery admins can temporarily lose effective recovery until governance installs a new default admin set.

## RECOMMENDATION

Re-run `LibRecovery.validateAdmins` against the current executor inside `executeDefaultRecoveryAdmins` before writing `_defaultRecoveryAdmins`.

Also guard executor rotation against known default recovery admins. If the pending executor equals an active or pending default recovery admin, require governance to update or cancel the recovery-admin configuration first. At minimum, emit a clear warning event or expose a view that reports default admins made ineffective by the current executor.

## RESOLUTION

Status: Acknowledged.

LOW **L-07**

ACKNOWLEDGED

# Allowance spends check relayer for removal

|          |                               |
|----------|-------------------------------|
| CATEGORY | Unexpected Behavior           |
| LOCATION | src/facets/AllowanceFacet.sol |
| REVIEW   | Remediation Review            |

## DESCRIPTION

`spendDailyAllowance` and `spendDailyAllowanceEth` try to enforce the pending-removal quarantine on the wrong address. After verifying the executor signature, both functions derive a `signerKey` from `msg.sender` and pass that key to `LibAuth.enforceNotPendingRemoval`.

That check does not identify a wallet signer. These spend functions are intentionally callable by anyone with a valid executor signature, so `msg.sender` is only the account submitting the transaction. The executor-signed payload binds the recipient, amount, nonce, deadline and executor epoch. It does not bind a wallet owner or passkey signer and these functions do not verify an owner signature.

For example, if owner `A` is pending removal and submits an executor-signed allowance spend directly, the check may reject because `msg.sender` maps to `A`. The same signed spend can be submitted by relayer `R`. In that case the contract checks `R` instead of `A` and the pending-removal quarantine does not apply.

Consequently, this check is only a misleading security control. The code appears to block pending-removal signers from triggering daily allowance spends, but no signer is actually tied to the spend authorization. The only meaningful authorization for these functions is the executor signature.

## RECOMMENDATION

Remove the pending-removal check from allowance spend functions if daily allowance spending is meant to be executor-only.

If pending-removal signers must be unable to trigger these spends, bind a signer key into the spend payload and verify an owner signature for that signer before the transfer. Then run

`LibAuth.enforceNotPendingRemoval` on the verified signer key, not on `msg.sender`.

## RESOLUTION

Status: Acknowledged.

LOW **L-08**

ACKNOWLEDGED

# Recipient blocklist is not wallet-wide

CATEGORY Access Control

LOCATION `src/facets/WithdrawalFacet.sol`, `src/facets/ExecutorExternalCallFacet.sol`, `src/facets/ExternalApproveFacet.sol`, `src/facets/ExternalApproveAndPreAuthorizeHashFacet.sol`

REVIEW Remediation Review

## DESCRIPTION

The wallet screens fund destinations against the BatchExecutor blocklist with `_enforceRecipientNotBlocked` (AllowanceFacet), but only calls it on the two allowance-spend paths (`spendDailyAllowance`, `spendDailyAllowanceEth`). Every other value or approval exit moves funds without it, so the blocklist is per-facet, not wallet-wide. A destination rejected on an allowance spend is still reachable through:

1. Executor withdrawals. `WithdrawalFacet.executorTokenWithdrawal` / `executorEthWithdrawal` send to an arbitrary `to` with no screen:

```
IERC20(token).safeTransfer(to, amount);
(bool success,) = to.call{value: amount}("");
```

1. Generic external calls. `ExecutorExternalCallFacet.executorExternalCall` / `executorApproveAndExternalCall` call any `target` with any `data` and native `value`, unscreened. Their only filter is a selector denylist that omits `setApprovalForAll` (`0xa22cb465`), `transfer`, `transferFrom`, and native `value`:

```
if (selector == IERC20.approve.selector
|| selector == bytes4(keccak256("increaseAllowance(address,uint256)"))
|| selector == bytes4(keccak256("approve(address,address,uint160,uint48)")))
revert ApprovalSelectorBlocked();
```

So an executor-signed `transfer(blocked, amount)`, a native send to a blocked target, or `setApprovalForAll(operator, true)` all pass.

1. Approval grants. `ExternalApproveFacet.externalApprove` and `ExternalApproveAndPreAuthorizeHashFacet.externalApproveAndPreAuthorizeHash` approve an arbitrary `spender` with no screen:

```
IERC20(token).forceApprove(spender, amount);
```

A blocked spender then pulls the approved amount, the same outflow the blocklist is meant to stop. All paths need a valid owner plus executor co-signature, so this is a policy completeness gap, not an unprivileged theft path.

**L-08** Recipient blocklist is not wallet-wide

CONTINUED

**— RECOMMENDATION**

Route every value and approval exit through one shared recipient-screen helper, and invoke it for the withdrawal `to`, the external-call `target` and token recipient, and the approval `spender` /operator.

**— RESOLUTION**

Status: Acknowledged.

LOW **L-09**

ACKNOWLEDGED

# ProxyAdmin delay check can brick the kill switch

CATEGORY DoS  
LOCATION src/UserProxyFactoryBeacon.sol  
REVIEW Remediation Review

## DESCRIPTION

The remediation added `_validateProxyAdminDelay` and wired it into four factory governance functions: `execzuteUpgrade`, `executeExecutorChange`, `executeFacetUpdate`, and `executeDefaultRecoveryAdmins`. The most sensitive is `executeExecutorChange`, the executor kill switch: it is the only function that increments `executorEpoch`, which is what invalidates a compromised executor's signed operations and standing pre-authorized orders. Operators reach for it precisely when the executor key is suspected compromised.

`_validateProxyAdminDelay` reads the ProxyAdmin from the ERC-1967 admin slot. If the ProxyAdmin is a contract it staticcalls `owner()`; if that owner is also a contract it staticcalls `getMinDelay()` and requires at least `UPGRADE_DELAY` (48h). When the owner has no `getMinDelay()` it reverts `ProxyAdminOwnerNotTimelock`; when the delay is too short it reverts `ProxyAdminDelayTooShort`. The check runs at the top of all four functions, so each reverts whenever the ProxyAdmin owner is a contract that is not a TimelockController.

This trigger is common and benign: the standard pattern owns a ProxyAdmin with a plain multisig such as a Gnosis Safe, which has no `getMinDelay()`, so the check reverts. In that state an operator cannot rotate the executor or bump `executorEpoch`, so during a compromise the executor's signed operations and standing orders cannot be invalidated on-chain. Upgrades, facet updates, and recovery-admin refreshes are blocked too.

This stays Low: there is no unprivileged attacker (the trigger is the operator's own privileged choice of ProxyAdmin owner), and it is recoverable by moving ProxyAdmin ownership to a TimelockController with a delay of 48h or more, though that recovery is itself a governance action and awkward mid-incident.

Note the escape hatch is inverted: an EOA owner returns early and passes, while a multisig owner reverts. So the least-safe shape (a single EOA) sails through, while hardening ProxyAdmin ownership to a Safe silently disables the kill switch. The check also does not close the original issue: a direct

`ProxyAdmin.upgradeAndCall` on the factory proxy never routes through these four functions, so the delay assertion does not constrain it.

**L-09** ProxyAdmin delay check can brick the kill switch

CONTINUED

**RECOMMENDATION**

Document that a multisig or Safe-owned ProxyAdmin (anything without a TimelockController-style `getMinDelay()`) makes `executeExecutorChange`, `executeUpgrade`, `executeFacetUpdate`, and `executeDefaultRecoveryAdmins` revert and disables the kill switch until ProxyAdmin ownership is moved to a 48h timelock, or treat a missing `getMinDelay()` as a skip rather than a hard revert.

**RESOLUTION**

Status: Acknowledged.

LOW **L-10**

ACKNOWLEDGED

## Batch subcall gas can starve later items

|          |                       |
|----------|-----------------------|
| CATEGORY | Unexpected Behavior   |
| LOCATION | src/BatchExecutor.sol |
| REVIEW   | Remediation Review    |

### — DESCRIPTION

`batchExecute` only checks whether `gasleft()` is below `GAS_RESERVE` before starting each item. It then forwards `gasleft() - GAS_RESERVE` to the wallet call. The fixed reserve protects some caller-side cleanup, but it still lets one successful item consume nearly all gas available to the shared batch.

This matters because wallet calls can make recipient-controlled calls. For example, `spendDailyAllowanceEth` sends ETH with `to.call{value: amount}("")`. An approved recipient can accept the payment and burn gas in `receive()` before returning successfully. Consequently, that item can settle while later valid items in the same batch fail with `false` or remain unattempted because the loop no longer has enough gas to continue.

### — RECOMMENDATION

Forward a bounded per-item gas amount instead of almost all remaining gas. The batch executor should also reserve enough gas for the remaining loop work, result writes, event emission and return data.

If some approved recipients are expected to be gas-heavy, isolate them from unrelated settlements or meter them through a separate batch policy so one successful payment cannot consume the gas budget for later items.

### — RESOLUTION

Status: Acknowledged.

LOW **L-11**

ACKNOWLEDGED

## Stale exits end later direct-mode sessions

|          |                                 |
|----------|---------------------------------|
| CATEGORY | Unexpected Behavior             |
| LOCATION | src/libraries/LibDirectMode.sol |
| REVIEW   | Remediation Review              |

### — DESCRIPTION

`InitiateExitDirectMode` signatures bind only the signer, nonce and deadline. They do not bind the active `directModeAt` timestamp. As a result, a valid but unused exit signature can be submitted during a later direct-mode session if the signer nonce has not changed and the deadline has not expired.

`DirectModeFacet.initiateExitDirectMode` verifies that stale signature, consumes the signer nonce and calls `LibDirectMode.initiateExitDirectMode`. That function only checks that the wallet is currently in direct mode and that no exit is already pending. It does not know which direct-mode session the signer meant to exit.

Consequently, any holder of the old signature can schedule an exit for a later direct-mode session. After the normal one-hour exit cooldown, `isDirectMode()` returns false for that later session. During a global pause, this can temporarily remove the user's direct-withdrawal escape option until the user cancels the pending exit or starts direct mode again after another cooldown.

### — RECOMMENDATION

Include the active `directModeAt` timestamp in the `InitiateExitDirectMode` EIP-712 payload and verify that it still matches wallet state before scheduling an exit.

Also consider storing enough session context for exits so `isDirectMode()` ignores a matured exit that was not created for the current direct-mode activation.

### — RESOLUTION

Status: Acknowledged.

LOW **L-12**

ACKNOWLEDGED

# No-op recovery config cancels pending recovery

|          |                              |
|----------|------------------------------|
| CATEGORY | Unexpected Behavior          |
| LOCATION | src/facets/RecoveryFacet.sol |
| REVIEW   | Remediation Review           |

## DESCRIPTION

`setRecoveryConfig` clears any active pending recovery before it rewrites the recovery admin list. It does this for every non-empty `admins` array and does not compare the submitted roster to the wallet's current effective recovery admins.

Consequently, an idempotent config update can act as a recovery cancellation. If an unused owner-signed `SetRecoveryConfig` payload still matches the current admin roster, submitting it during an active recovery consumes the signer nonce and clears `rs.pendingActive`, even though the recovery configuration did not actually change.

This bypasses the intent-specific binding used by `cancelRecovery`. `CancelRecovery` signs the current pending recovery target through `pendingNewSignerHash`, while `SetRecoveryConfig` only signs `signer`, `configHash`, `nonce` and `deadline`. A holder of one valid no-op config-update signature can therefore delay recovery once without a `CancelRecovery` signature. The impact is bounded to liveness grieving because the nonce is consumed and recovery admins can initiate recovery again.

## RECOMMENDATION

Only clear pending recovery when the effective recovery admin roster actually changes. Compare the submitted admins against `LibRecovery.getEffectiveAdmins` before calling `LibRecovery.clearPendingRecovery`.

If config updates are intended to cancel recovery even when the roster is unchanged, bind that intent explicitly into the signed data. For example, include the active recovery identifier or `pendingNewSignerHash` in the signed config-update payload when cancellation is allowed.

## RESOLUTION

Status: Acknowledged.

LOW **L-13**

ACKNOWLEDGED

## Beacon wallets reject 2300-gas native transfers

|          |                                |
|----------|--------------------------------|
| CATEGORY | Unexpected Behavior            |
| LOCATION | src/UserProxyFactoryBeacon.sol |
| REVIEW   | Remediation Review             |

### — DESCRIPTION

User wallets are deployed as OpenZeppelin `BeaconProxy` contracts. The proxy has no cheap `receive()` function of its own. Therefore, an empty-calldata native AVAX transfer to a wallet proxy enters the OpenZeppelin proxy fallback, calls the beacon to read the current implementation and then delegatecalls into `WalletDiamond.receive()`.

That work costs more than the 2300 gas stipend forwarded by Solidity `transfer()` and `send()`. Third-party contracts that hardcode those send methods for AVAX refunds or payouts will fail when the recipient is an EthenaPay wallet. This does not block native deposits made with `call{value:}` and sufficient gas and the core wallet withdrawal code already uses low-level calls for native transfers.

A focused Foundry test confirmed the behavior on a real deployed wallet proxy from the project fixture: `transfer()` reverted, `send()` returned false and `call{value:, gas: 100_000}` funded the wallet.

### — RECOMMENDATION

Document that native AVAX senders must use `call{value:}` with enough gas when sending to EthenaPay wallets.

If compatibility with `transfer()` and `send()` is required, deploy wallets through a proxy shell that implements a cheap proxy-level `receive()` function and stores the funds without performing a beacon lookup or delegatecall.

### — RESOLUTION

Status: Acknowledged.

LOW **L-14**

ACKNOWLEDGED

# ECDSA bypasses code-owner ERC-1271 policy

|          |                           |
|----------|---------------------------|
| CATEGORY | Validation                |
| LOCATION | src/libraries/LibAuth.sol |
| REVIEW   | Remediation Review        |

## DESCRIPTION

`LibAuth.verifyOwnerSignature` lets the caller choose the signature type with the first byte of `ownerSig`. When that byte is `0x00`, `_verifyECDSA` recovers an address and accepts the signature if the recovered address is a registered owner. It does not reject owners that currently have code.

The ERC-1271 verifier handles the same address class differently. For `0x01` signatures, `_verifyEIP1271` requires the owner address to have code and then calls `SignatureChecker.isValidSignatureNow`. Consequently, a code-bearing owner can reject an operation through its ERC-1271 policy but still authorize the same wallet operation with a raw secp256k1 signature.

This matters for EIP-7702-style delegated EOAs if their contract code is meant to define the active signing policy. In that model, compromise or unintended use of the raw EOA key bypasses the delegated account's `isValidSignature` policy for EthenaPay operations.

## RECOMMENDATION

Persist the owner kind at registration and enforce the matching signature type during verification. Code-bearing owners that are registered as smart-contract owners should authenticate through ERC-1271.

As a smaller hardening change, reject ECDSA verification when the recovered registered owner currently has code. If raw EOA authority is intentionally trusted for delegated EOAs, document that behavior explicitly so integrators do not assume `isValidSignature` is authoritative for those owners.

## RESOLUTION

Status: Acknowledged.

LOW **L-15**

ACKNOWLEDGED

# Pause probe blocks direct-mode signer changes

|          |                                 |
|----------|---------------------------------|
| CATEGORY | Validation                      |
| LOCATION | src/libraries/LibDirectMode.sol |
| REVIEW   | Remediation Review              |

## DESCRIPTION

`enforceUserNotBlocked()` calls `isGlobalPaused()` before it checks whether the wallet is already in direct mode. `isGlobalPaused()` intentionally fails closed and reverts with `GlobalPauseActive` when the factory `globalPaused()` probe reverts or returns malformed data. Therefore, a broken pause probe prevents the direct-mode exemption from being evaluated.

This contradicts the documented direct-mode guarantee for signer management. With a working `globalPaused() == true` probe, an already-direct-mode wallet can still approve signer removal. With only `globalPaused()` mocked to revert, the same direct-mode signer-removal operation reverts with `GlobalPauseActive`.

The impact is limited to the modeled factory-probe outage. Direct withdrawals still work in direct mode and `revokePreAuthorizedHash()` is not affected in the current code because it no longer calls `enforceUserNotBlocked()`. The remaining issue is that users can lose active signer-management controls, such as finalizing removal of a compromised signer, until the factory pause probe is repaired.

## RECOMMENDATION

Check local direct-mode state before probing the factory for user-gated operations. For example:

```
function enforceUserNotBlocked() internal view {  
    if (isDirectMode()) return;  
    if (isGlobalPaused()) revert GlobalPauseActive();  
}
```

Alternatively, use a non-reverting pause-probe helper for this guard and keep the fail-closed behavior only where direct mode is not supposed to override global pause.

---

**L-15** Pause probe blocks direct-mode signer changes

CONTINUED

---

— **RESOLUTION**

Status: Acknowledged.

LOW **L-16**

ACKNOWLEDGED

# Stale pending owner additions outlive removal

|          |                           |
|----------|---------------------------|
| CATEGORY | Unexpected Behavior       |
| LOCATION | src/libraries/LibAuth.sol |
| REVIEW   | Remediation Review        |

## DESCRIPTION

`executeRemoveOwner` removes an address owner but does not clear pending owner additions for the same address. This matters because `addOwnerDirect` can add an address through recovery while an existing pending owner addition for that address is still live.

For example, a recovery admin can initiate recovery, then a current signer can initiate a normal owner addition for the same address late enough in the recovery delay that the pending addition remains within its 7-day TTL. When recovery later executes, `addOwnerDirect` adds the address but leaves the pending addition in storage. `approveAddOwner` is blocked only while the address is still an owner, so the pending addition remains retryable. If that owner is removed before the pending addition expires, a current signer can approve the old action immediately and re-add the removed owner without a fresh 24-hour initiation period.

## RECOMMENDATION

Invalidate pending owner additions that target an address whenever that address is removed. Also clear any matching pending owner addition inside `addOwnerDirect` before or after adding the recovered owner.

Alternatively, track a per-owner generation counter and bind pending owner additions to that generation. Increment the generation on removal so older pending additions cannot be approved after the address is removed.

## RESOLUTION

Status: Acknowledged.

LOW **L-17**

ACKNOWLEDGED

## Same-block direct-mode cancels collide

|          |                                 |
|----------|---------------------------------|
| CATEGORY | Unexpected Behavior             |
| LOCATION | src/libraries/LibDirectMode.sol |
| REVIEW   | Remediation Review              |

### DESCRIPTION

`CancelDirectMode` and `CancelExitDirectMode` signatures are meant to cancel one specific pending direct-mode action. The signed payload includes the signer nonce, deadline and the pending activation timestamp currently stored in the wallet. It does not include a unique action id or the digest of the action that the signer intended to cancel.

That timestamp is not unique. `initiateDirectMode` sets `directModeAt` to `block.timestamp + 1 hours` and `initiateExitDirectMode` sets `exitDirectModeAt` the same way. If a pending action is cancelled and then recreated in the same block, the new action receives the same pending timestamp as the old one.

As a result, an unused cancel signature for the old pending action can also cancel the freshly recreated action. Anyone holding that signature can submit it, consume the signer's nonce and clear the new direct-mode enter or exit request.

For example:

- At timestamp `T`, signer A schedules direct mode, so `directModeAt = T + 1 hours`.
- Signer B signs a cancellation for that pending direct-mode request, but the signature is not submitted immediately.
- In the same block, the first request is cancelled and a new direct-mode request is created.
- Because the block timestamp is still `T`, the new request also stores `directModeAt = T + 1 hours`.
- The old cancellation signature now matches the new request and can cancel it even though the signer approved cancelling the previous request.

The impact is temporary liveness griefing, not fund loss. The stale cancel is one-shot because submitting it consumes the signer's nonce. The collision also disappears once a later block gives the recreated action a different timestamp.

---

**L-17** Same-block direct-mode cancels collideCONTINUED

---

**— RECOMMENDATION**

Bind cancel signatures to a value that uniquely identifies the pending action. A monotonic direct-mode action nonce is the simplest fix. Increment it whenever `initiateDirectMode` or `initiateExitDirectMode` creates a new pending action, include that action nonce in the corresponding cancel EIP-712 payload and verify that it still matches storage before clearing the pending state.

As a weaker operational mitigation, avoid reinitiating direct-mode enter or exit in the same block after cancellation. This avoids the timestamp collision, but it relies on caller behavior and should not replace protocol-level action binding.

**— RESOLUTION**

Status: Acknowledged.

LOW **L-18**

ACKNOWLEDGED

## Zero-coordinate P-256 passkeys are rejected

|          |                           |
|----------|---------------------------|
| CATEGORY | Validation                |
| LOCATION | src/libraries/LibAuth.sol |
| REVIEW   | Remediation Review        |

### — DESCRIPTION

Passkey registration rejects any P-256 public key whose `qx` or `qy` coordinate is zero before it asks `P256.isValidPublicKey` whether the point is on the curve. This excludes mathematically valid secp256r1 public keys. For example, OpenZeppelin's `P256.isValidPublicKey` accepts a real point with `qx == 0`, but the wallet rejects it first with `InvalidPublicKey`.

The same zero-coordinate precheck is repeated in `LibAuth.initiateAddPasskey`, `LibAuth.addPasskeyDirect` and `RecoveryFacet.initiateRecovery`. The factory deployment functions also reject zero passkey coordinates before the wallet initializer runs.

Consequently, a user whose authenticator produces a valid WebAuthn key with a zero coordinate cannot use that passkey for wallet initialization, signer addition or recovery. This is a low-likelihood compatibility issue rather than a theft vector, because random authenticator-generated keys should almost never hit this edge case.

### — RECOMMENDATION

Remove the blanket zero-coordinate rejection for P-256 passkeys and rely on `P256.isValidPublicKey` to decide whether the submitted coordinates are valid.

Keep the existing duplicate checks, signer-count checks and format checks. Apply the same validation rule consistently in wallet initialization, passkey addition, direct recovery add and `RecoveryFacet.initiateRecovery`.

### — RESOLUTION

Status: Acknowledged.

LOW **L-19**

ACKNOWLEDGED

## Zero externalApprove records ghost owner

|          |                                     |
|----------|-------------------------------------|
| CATEGORY | Unexpected Behavior                 |
| LOCATION | src/facets/ExternalApproveFacet.sol |
| REVIEW   | Remediation Review                  |

### — DESCRIPTION

`externalApprove` records a deadline-gated approval entry even when `amount` is zero. The function first clears the ERC-20 allowance with `forceApprove(spender, 0)`, but then it still creates a new `approveHash`, stores an approval entry and marks that hash as the current owner for the `(token, spender)` slot.

Consequently, `currentApprovalOwner` and `approvalEntry` can report a live approval owner even though the token allowance is already zero. Permissionless cleanup through `clearStaleApproval` is also unavailable until `approvalDeadline`, so the stale metadata remains visible until the deadline passes.

This does not let the spender pull funds. A later tracked approval for the same token and spender can also replace the ghost owner. The impact is misleading wallet accounting and delayed metadata cleanup, not direct fund loss.

### — RECOMMENDATION

Special-case zero-amount approvals in `externalApprove`.

After `forceApprove(spender, 0)`, clear the current owner for the `(token, spender)` slot and skip `setEntry` for the zero approval. Only create a new deadline-gated approval entry when `amount` is nonzero.

### — RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-01**

ACKNOWLEDGED

# Emergency revocations emit no events

|          |                                          |
|----------|------------------------------------------|
| CATEGORY | Events                                   |
| LOCATION | src/facets/ExecutorExternalCallFacet.sol |
| REVIEW   | Remediation Review                       |

## DESCRIPTION

`emergencyPermit2Lockdown` and `emergencyRevokeApproval` perform emergency cleanup but emit no dedicated event. `emergencyPermit2Lockdown` calls `Permit2Lockdown` and `emergencyRevokeApproval` can revoke a preauthorized hash, clear deadline-gated approval metadata and set an ERC-20 allowance to zero. None of those owner-only emergency actions produces a wallet-level event that clearly identifies the cleanup.

This makes incident response and monitoring weaker than the normal approval flows. Indexers can observe token `Approval` events or `Permit2`-specific events if those integrations emit them, but the wallet does not emit a canonical record that an emergency owner-authorized cleanup was executed. Off-chain systems therefore need protocol-specific decoding and can miss the user-protective action at the wallet layer.

## RECOMMENDATION

Emit dedicated wallet events for both emergency functions. The events should include the verified signer key, the affected token and spender or `Permit2` address and enough metadata to distinguish the cleanup from normal executor-approved external calls.

For `Permit2` lockdowns, emit the `Permit2` address and a hash of the approval list. For ERC-20 revocations, emit the token, spender and any preauthorized hash that was revoked.

## RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-02

ACKNOWLEDGED

# Direct NFT transfers lack target checks

|          |                                |
|----------|--------------------------------|
| CATEGORY | Warning                        |
| LOCATION | src/facets/WithdrawalFacet.sol |
| REVIEW   | Remediation Review             |

## DESCRIPTION

`directNFTTransfer` and `directERC1155Transfer` reject zero token-contract addresses, but they do not check that the supplied address contains code or supports the expected token interface before consuming the signer nonce.

For a normal ERC-721 or ERC-1155 token, the token contract enforces ownership and transfer rules. The problem is malformed input. If a signer supplies an EOA or an unrelated contract with a permissive fallback as the token address, the wallet can consume the direct-mode nonce and emit a direct transfer event even though no NFT or ERC-1155 balance moved.

## RECOMMENDATION

Before consuming the nonce, require the token address to contain code. Consider also checking ERC-165 support for the expected interface, using `type(IERC721).interfaceId` for `directNFTTransfer` and `type(IERC1155).interfaceId` for `directERC1155Transfer`.

If compatibility with non-ERC165 tokens is required, at least reject `tokenContract.code.length == 0` and document that interface validation is intentionally delegated to the token call.

## RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-03

ACKNOWLEDGED

# Base64URL challenge encoding overflows output

|          |                                                       |
|----------|-------------------------------------------------------|
| CATEGORY | Unexpected Behavior                                   |
| LOCATION | lib/openzeppelin-contracts/contracts/Utils/Base64.sol |
| REVIEW   | Remediation Review                                    |

## DESCRIPTION

`Base64._encode` computes a shorter output length when URL-safe Base64 is requested without padding. For a 32-byte input, the declared result length is 43 bytes. The encoder loop still writes every input chunk as a full four-character quartet, so the final partial chunk writes a 44th byte at the current free-memory pointer.

Ethena Pay reaches this code during WebAuthn verification. `LibAuth._verifyWebAuthn` converts the signed digest into a 32-byte challenge, then OpenZeppelin's `WebAuthn._validateChallenge` calls `Base64.encodeURL(challenge)` while checking the client data JSON. Consequently, each passkey verification executes the out-of-bounds memory write before continuing with normal challenge comparison and P-256 signature checks.

The observed write is currently overwritten by the next allocation in the surrounding WebAuthn challenge comparison. A local test confirmed the stray byte and also confirmed that the next allocation clears it. No authentication bypass, fund loss or passkey lockout was demonstrated. The issue is still worth fixing because the encoder violates Solidity memory-safety assumptions and leaves the result dependent on nearby allocation behavior.

## RECOMMENDATION

Patch or upgrade the vendored Base64 encoder so unpadding URL-safe encoding only writes the declared number of output bytes. The final partial input block should write two or three output bytes as appropriate, not a full four-byte quartet. Alternatively, keep the padded quartet in allocated memory and return a correctly trimmed object without leaving bytes outside the returned string.

## RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-04

ACKNOWLEDGED

# Factory init optimizes out EIP-1153 probe

|          |                                |
|----------|--------------------------------|
| CATEGORY | Unexpected Behavior            |
| LOCATION | src/UserProxyFactoryBeacon.sol |
| REVIEW   | Remediation Review             |

## DESCRIPTION

`initialize()` is documented as probing EIP-1153 support by executing `tload(0)`. The result is assigned to a local assembly variable that is never consumed. With the configured optimized via-IR Cancun build, the compiler removes the unused probe from the factory runtime bytecode.

This breaks the documented CC-EIP1153/T2-G deployment check. Operators and auditors can believe the factory independently verifies transient-storage support during initialization, but the optimized factory runtime contains no executable `TLOAD` or `TSTORE`. This matters because wallet facets still depend on `LibReentrancyGuard`, which uses transient storage for guarded user operations.

The practical impact is limited. Current production artifacts still contain other Cancun-only opcodes such as `MCOPY`, so a standard pre-Cancun deployment is expected to fail before the factory is usable. Therefore this does not demonstrate a trapped-funds scenario. The issue is that the stated fail-fast compatibility gate is not actually present as a separate runtime check.

## RECOMMENDATION

Make the probe observable so the optimizer cannot remove it. For example, write and read a transient-storage sentinel, then consume the loaded value in a condition that cannot be proven dead by the optimizer.

## RESOLUTION

Status: Acknowledged.

## INFORMATIONAL I-05

ACKNOWLEDGED

# Batch partial failures lack item log attribution

|          |                       |
|----------|-----------------------|
| CATEGORY | Events                |
| LOCATION | src/BatchExecutor.sol |
| REVIEW   | Remediation Review    |

## DESCRIPTION

`batchExecute` returns a `bool[]` result array that identifies which batch items succeeded and failed. The same function only emits `BatchExecuteCompleted` with aggregate `requested`, `attempted` and `succeeded` counts.

Transaction receipts include logs, but they do not expose Solidity return data. Consequently, a monitor that relies only on `BatchExecutor` logs can detect that a batch partially failed, but it cannot identify which item indices failed from the event alone.

## RECOMMENDATION

Emit per-item result data from `batchExecute` or include a compact bitmap/hash of the `results` array in `BatchExecuteCompleted`. The emitted data should let logs-only monitors map each attempted item index to its success or failure status without relying on return-data capture or trace tooling.

## RESOLUTION

Status: Acknowledged.

## § 05

# Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

# About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

**\$45B+**DIGITAL ASSETS UNDER  
REVIEW**200+**

ENGAGEMENTS COMPLETED

**500+**CRITICAL & HIGH FINDINGS  
REPORTED

Reach us at [audits@guardianaudits.com](mailto:audits@guardianaudits.com).